

第七章 图

第一讲

7.1 图的定义与术语

7.2 图的存储结构

练习与作业

第二讲

7.3 图的遍历

7.4 图的连通性问题

练习与作业

第三讲

7.5 有向无环图及其应用

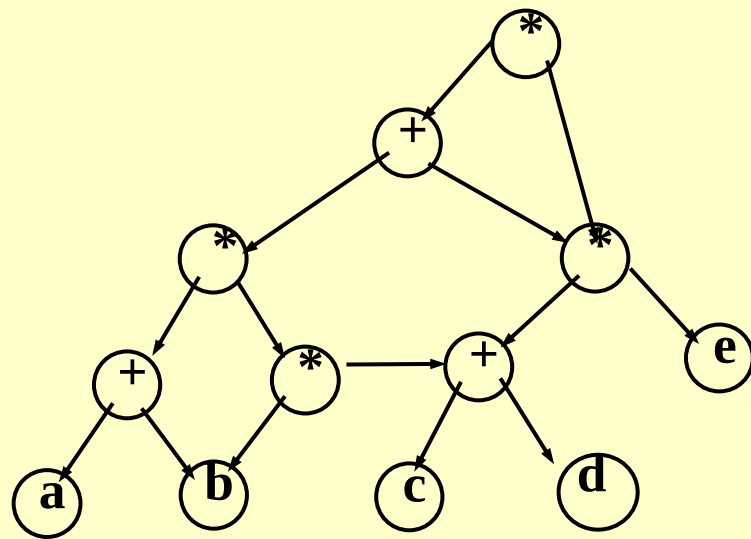
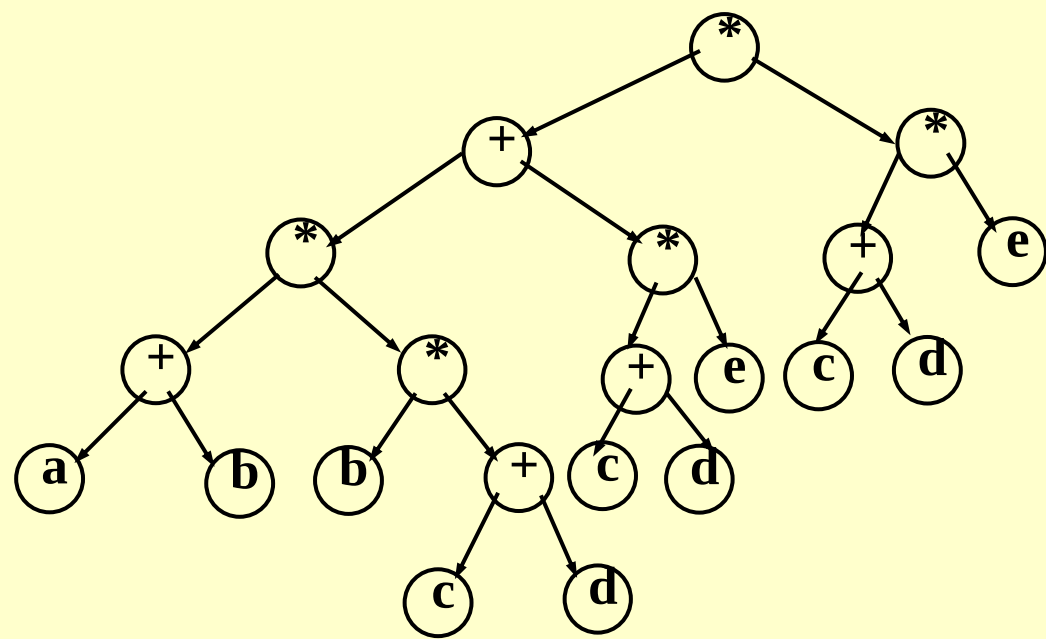
7.6 最短路径

练习与作业

有向无环图的概念:

- 一个无环的有向图称做有向无环图（**Directed Acycline Graph**）。简称**DAG图**。
- 应用：表达式的实现

$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$



有向无环图是描述一项工程或系统的进行过程的有效工具

一、拓扑排序:偏序关系集合上寻找全序

- 偏序与全序

- 若集合A中的二元关系R是自反的、非对称的和传递的，则R是A上的偏序关系。集合A与关系R一起称为一个偏序集合。（部分之间）
- 若R是集合A上的一个偏序关系，如果对每个 $a, b \in A$ 必有 aRb 或 bRa ，则R是A上的全序关系。集合A与关系R一起称为一个全序集合。（全体成员都可比较）

★问题提出：学生选修课程问题

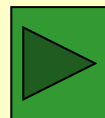
课程代号	课程名	先行课程代号	课程代号	课程名	先行课程代号
C1	程序设计导论	无	C8	算法分析	C3
C2	数值分析	C1, C13	C9	高级语言	C3, C4
C3	数据结构	C1, C13	C10	编译系统	C9
C4	汇编语言	C1, C12	C11	操作系统	C10
C5	自动机理论	C13	C12	解析几何	无
C6	人工智能	C3	C13	微积分	C12
C7	机器原理	C13			

■ 问题要求：用有向图表示

- 顶点——表示课程
- 有向弧——表示先决条件，若课程 i 是课程 j 的先决条件，则图中有弧 $\langle i, j \rangle$
- 学生应按怎样的顺序学习这些课程，才能无矛盾、顺利地完成学业——拓扑排序

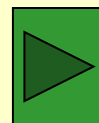
■ 定义

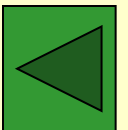
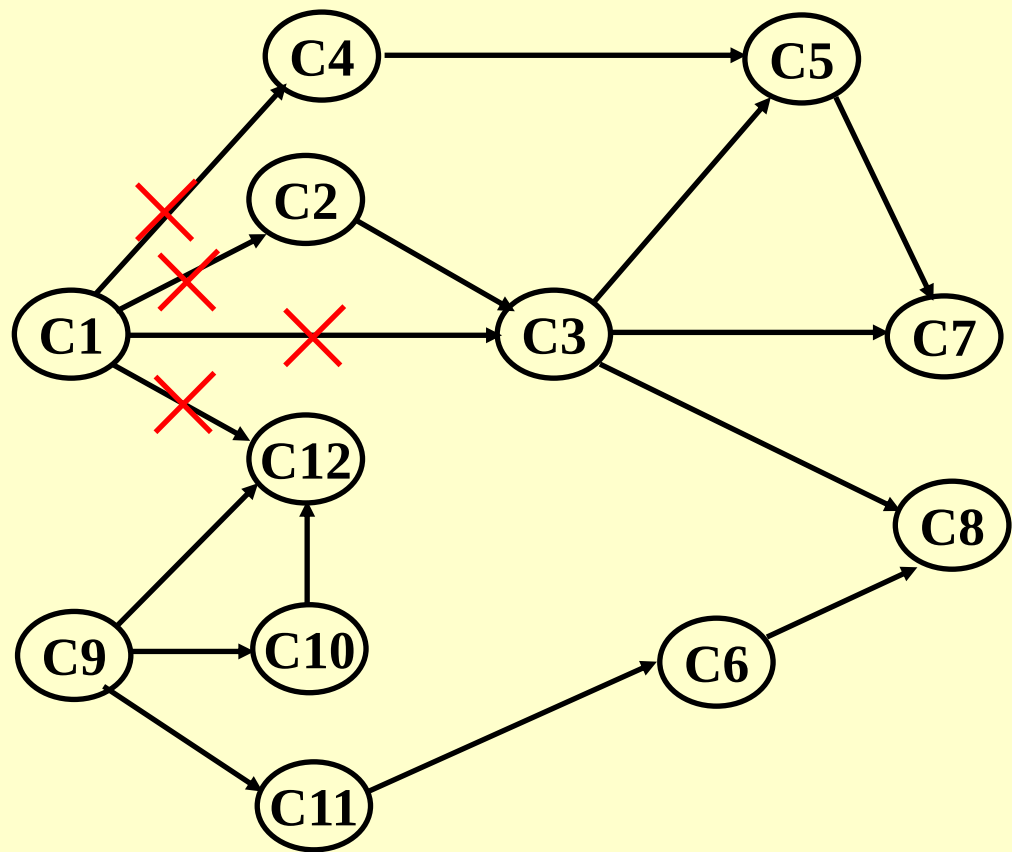
- **AOV网**——用顶点表示活动，用弧表示活动间优先关系的有向图称为顶点表示活动的网(Activity On Vertex network)，简称AOV网
 - 若 $\langle v_i, v_j \rangle$ 是图中有向边，则 v_i 是 v_j 的直接前驱； v_j 是 v_i 的直接后继
 - **AOV网中不允许有回路**，某项活动以自己为先决条件

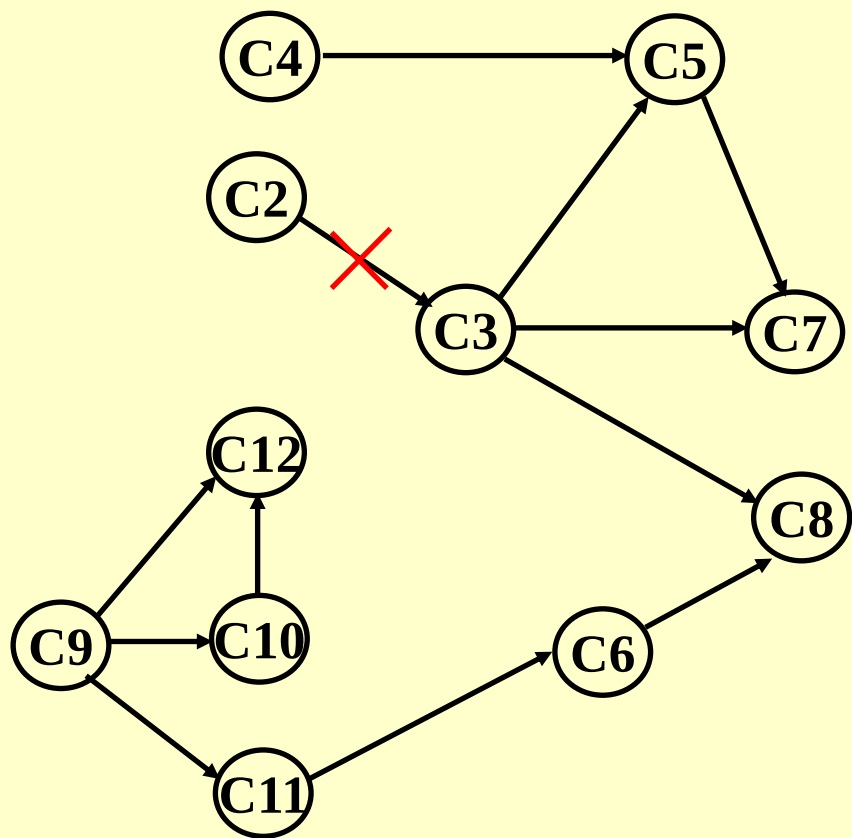


- **拓扑排序**——把AOV网络中各顶点按照它们相互之间的优先关系排列成一个线性序列的过程
- **检测AOV网中是否存在环方法：**

对有向图构造其顶点的拓扑有序序列，若网中所有顶点都在它的拓扑有序序列中，则该AOV网必定不存在环。
- **拓扑排序的方法**
 - 在有向图中选一个没有前驱的顶点且输出之；
 - 从图中删除该顶点和所有以它为尾的弧；
 - 重复上述两步，直至全部顶点均已输出、或者当图中不存在无前驱的顶点为止

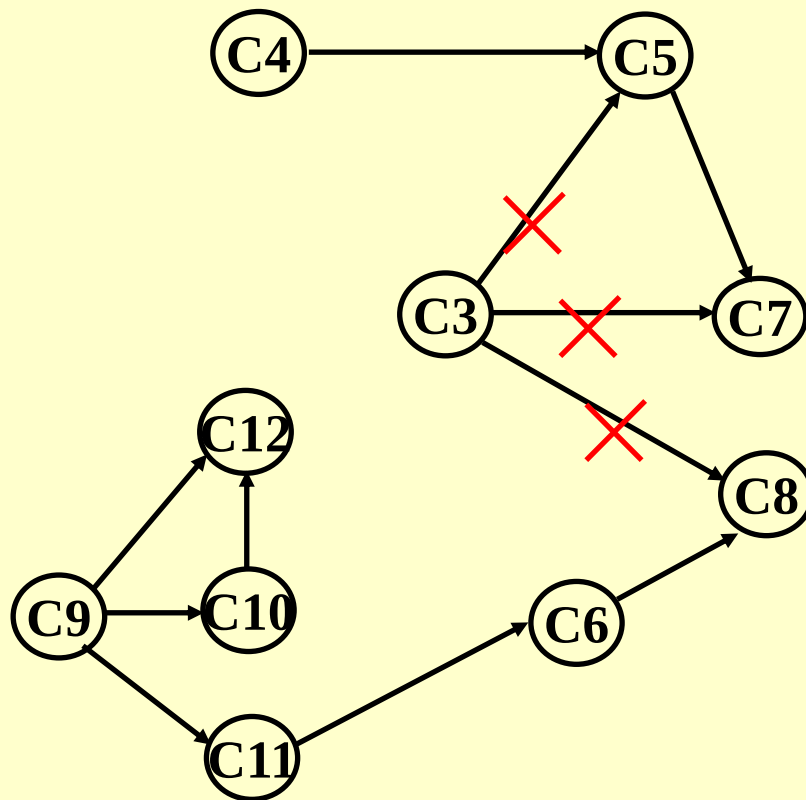






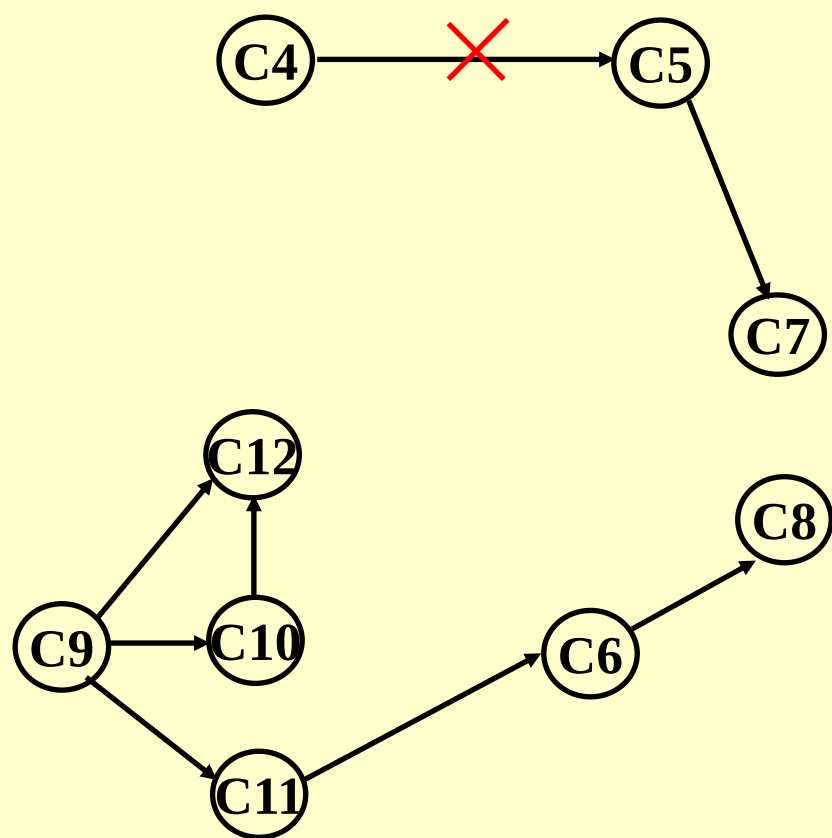
拓扑序列: C1

(1)

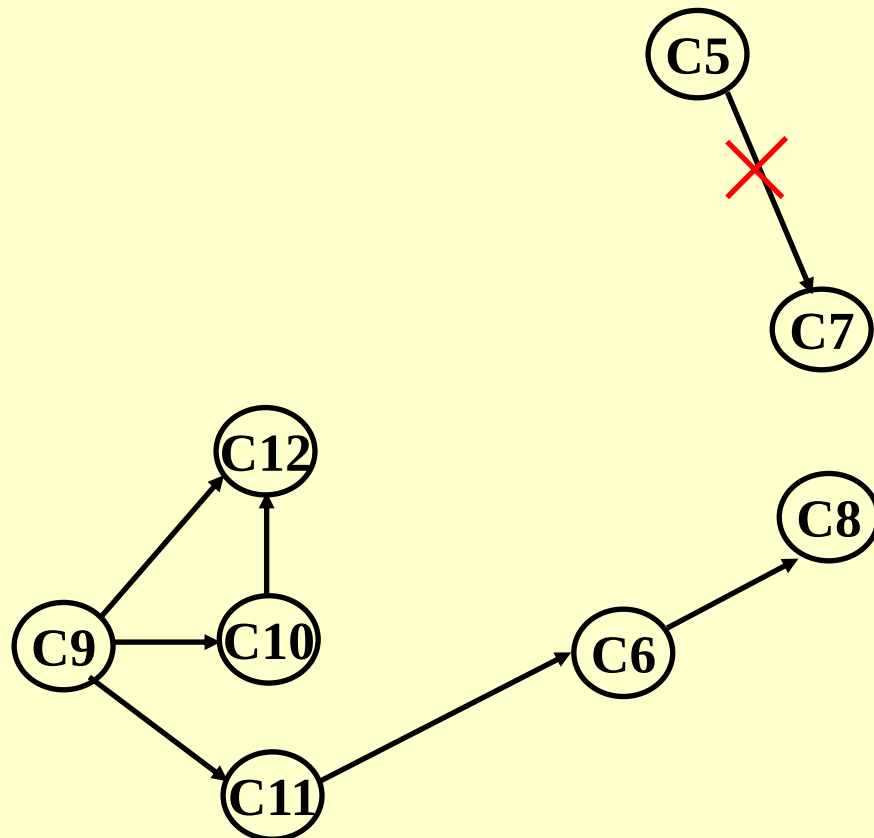


拓扑序列: C1--C2

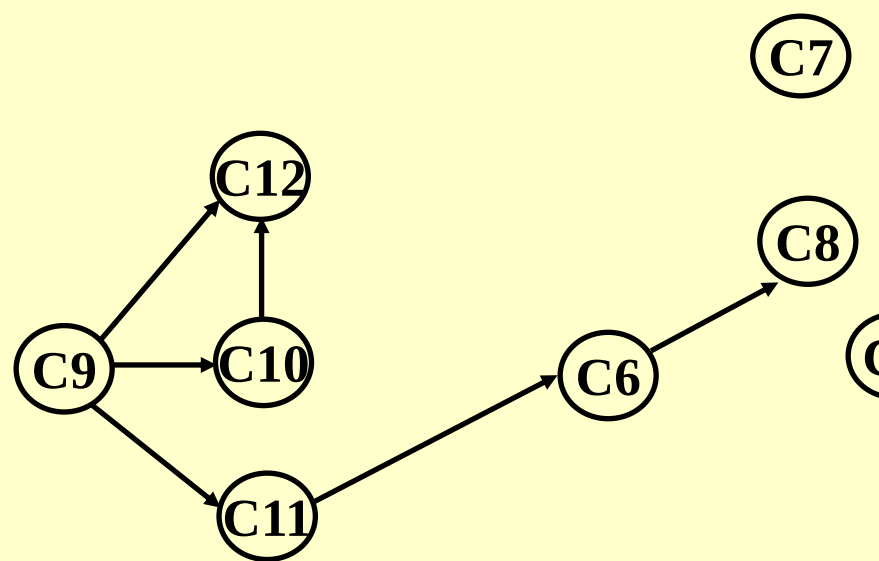
(2)



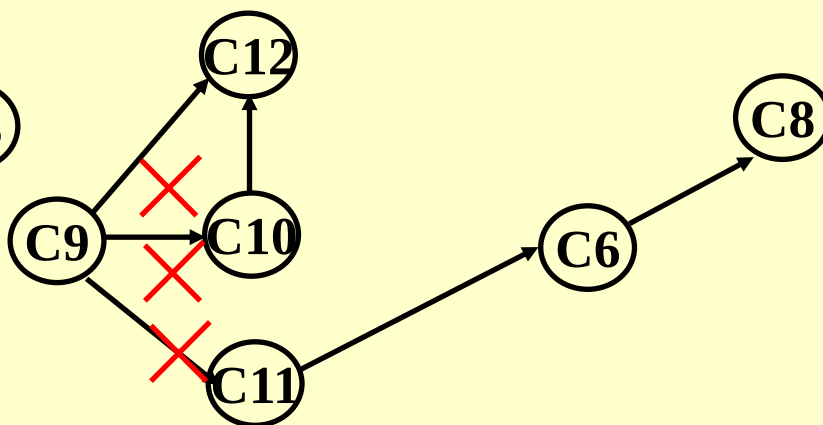
拓扑序列: C1--C2--C3
(3)



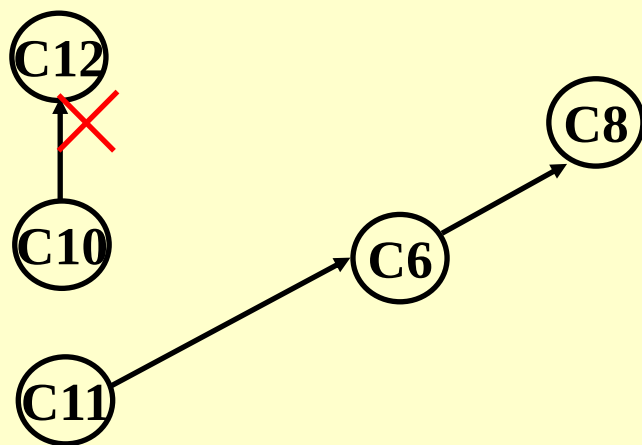
拓扑序列: C1--C2--C3--C4
(4)



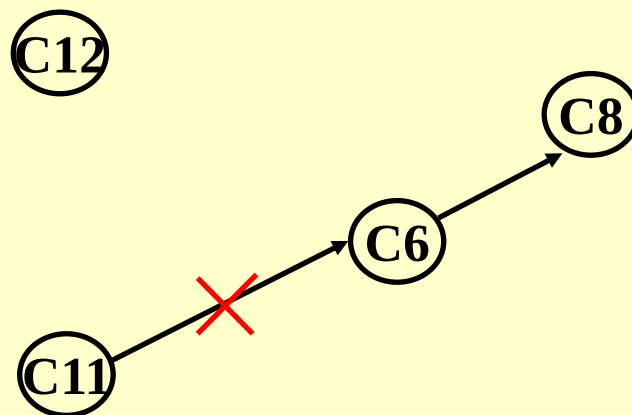
拓扑序列: C1--C2--C3--C4--C5
(5)



拓扑序列: C1--C2--C3--C4--C5--C7
(6)



拓扑序列: C1--C2--C3--C4--C5--C7--C9



拓扑序列: C1--C2--C3--C4--C5--C7--C9
--C10
(8)

C12

C8

C6



C12

C8

(9)

拓扑序列: C1--C2--C3--C4--C5--C7--C9
--C10--C11

(10)

拓扑序列: C1--C2--C3--C4--C5--C7--C9
--C10--C11--C6

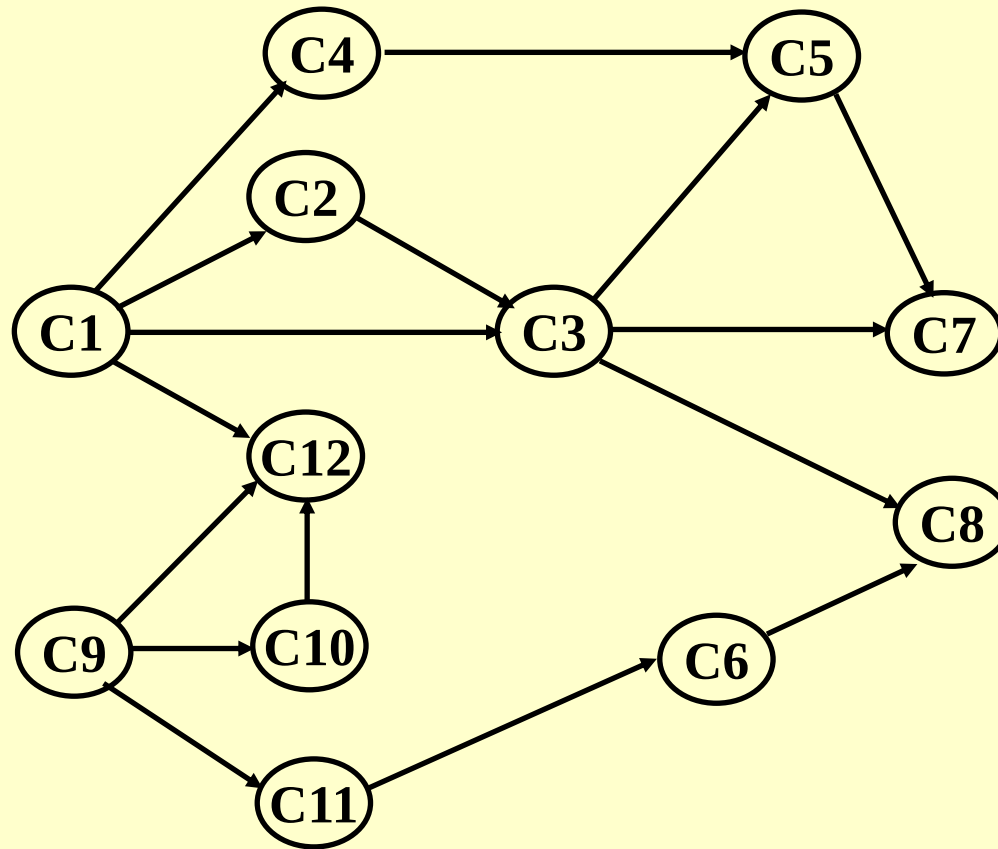
C8

(11)

拓扑序列: C1--C2--C3--C4--C5--C7--C9
--C10--C11--C6--C12

拓扑序列: C1--C2--C3--C4--C5--C7--C9
--C10--C11--C6--C12--C8

(12)



拓扑序列: C1--C2--C3--C4--C5--C7--C9--C10--C11--C6--C12--C8
或 : C9--C10--C11--C6--C1--C12--C4--C2--C3--C5--C7--C8

- 逻辑上一个AOV网的拓扑序列不是唯一的
- 当存储结构与访问次序确定后结果是唯一

拓扑排序的方法

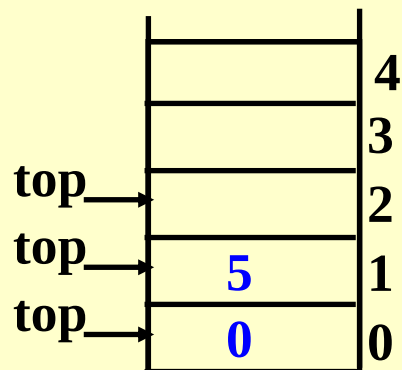
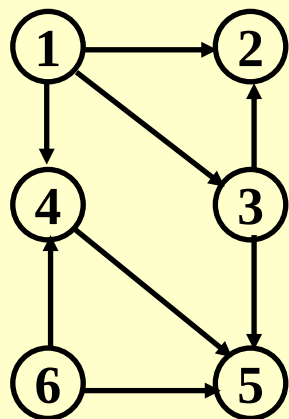
- 在有向图中选一个没有前驱的顶点且输出之
- 从图中删除该顶点和所有以它为尾的弧
- 重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止

● 算法实现

- 以邻接表为存储结构
- 把邻接表中所有入度为0的顶点进栈
- 栈非空时，输出栈顶元素 V_j 并退栈；在邻接表中查找 V_j 的直接后继 V_k ，把 V_k 的入度减1；若 V_k 的入度为0则进栈
- 重复上述操作直至栈空为止。若栈空时输出的顶点个数不是 n ，则有向图有环；否则，拓扑排序完毕。

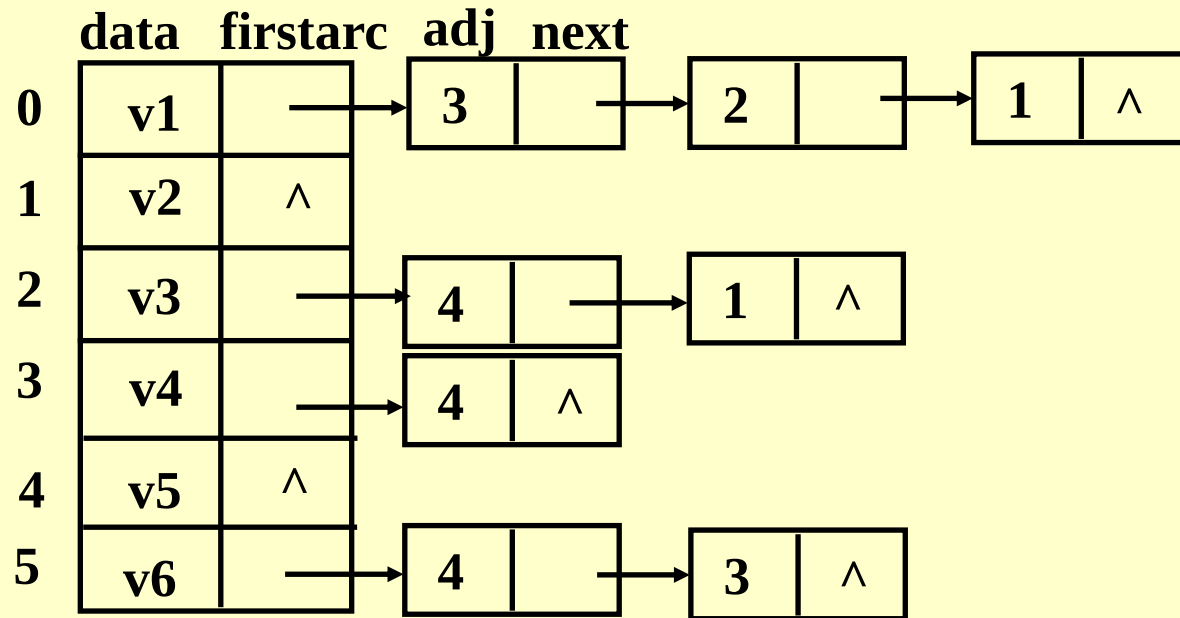
● 算法描述

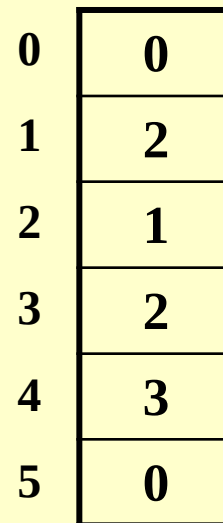
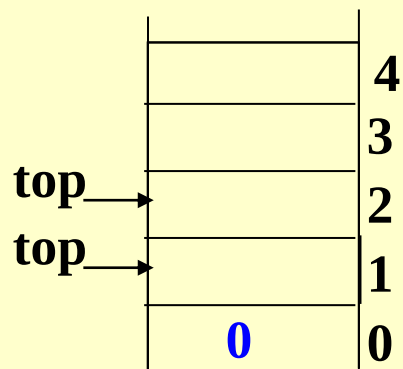
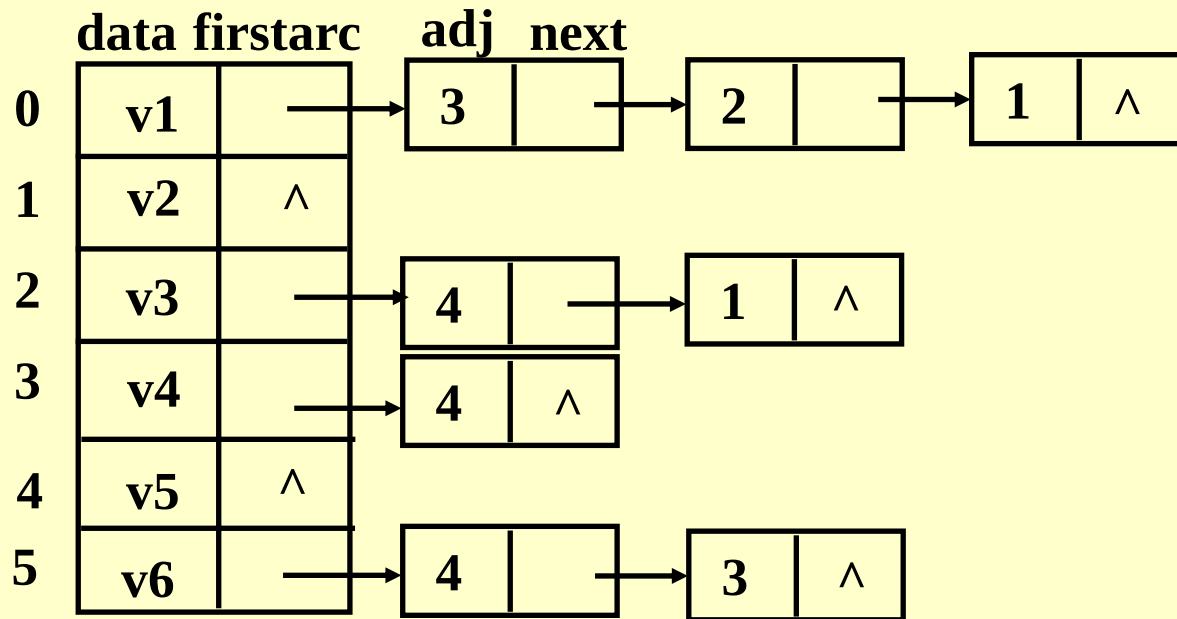
例



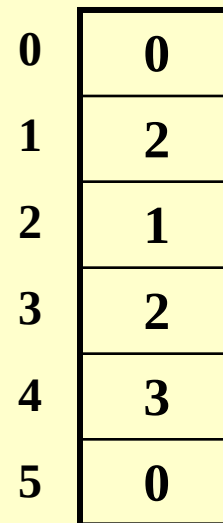
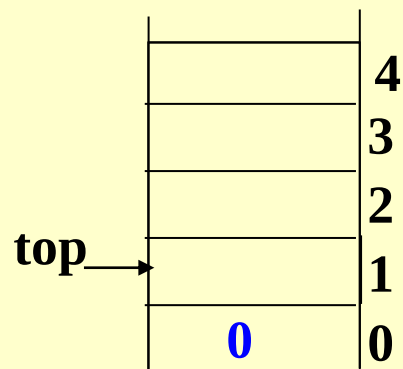
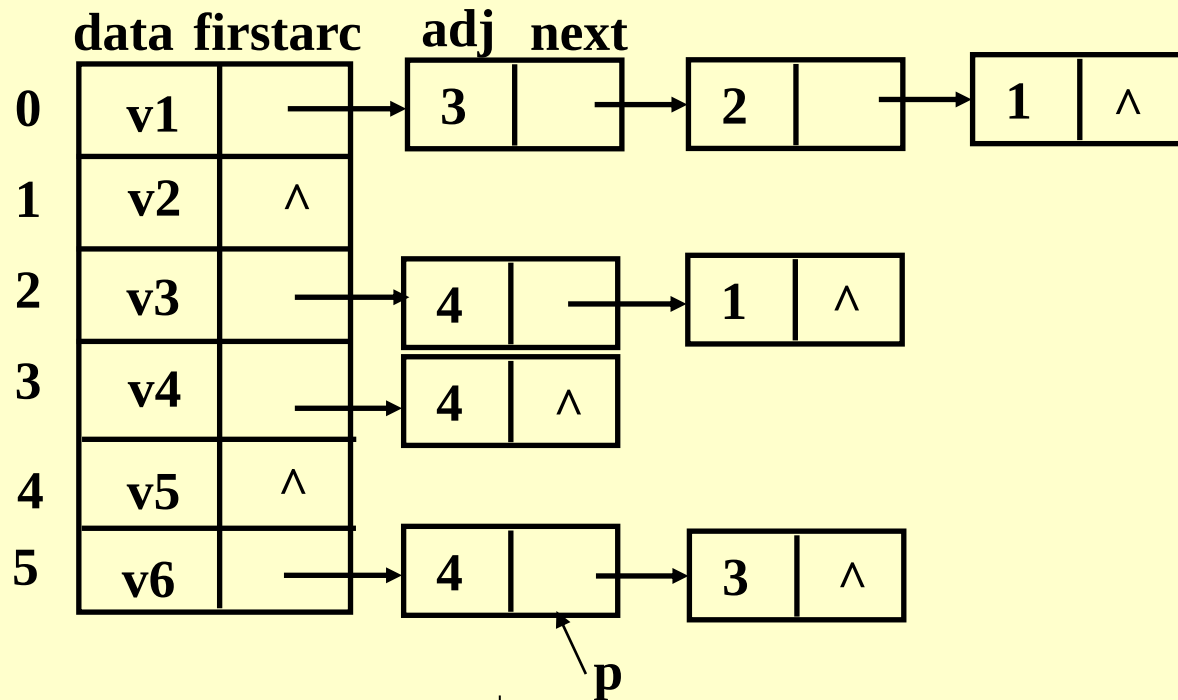
0	0
1	2
2	1
3	2
4	3
5	0

Indegree[]

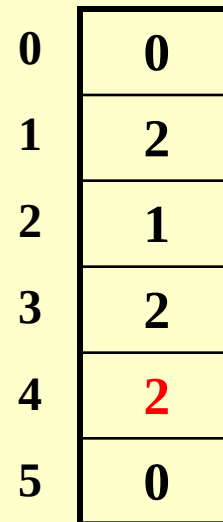
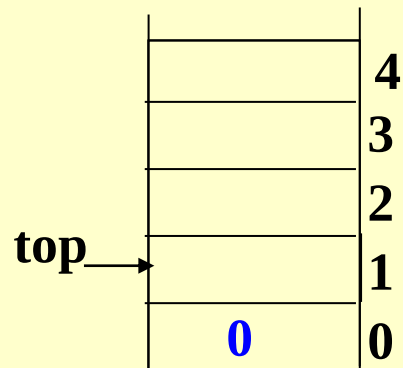
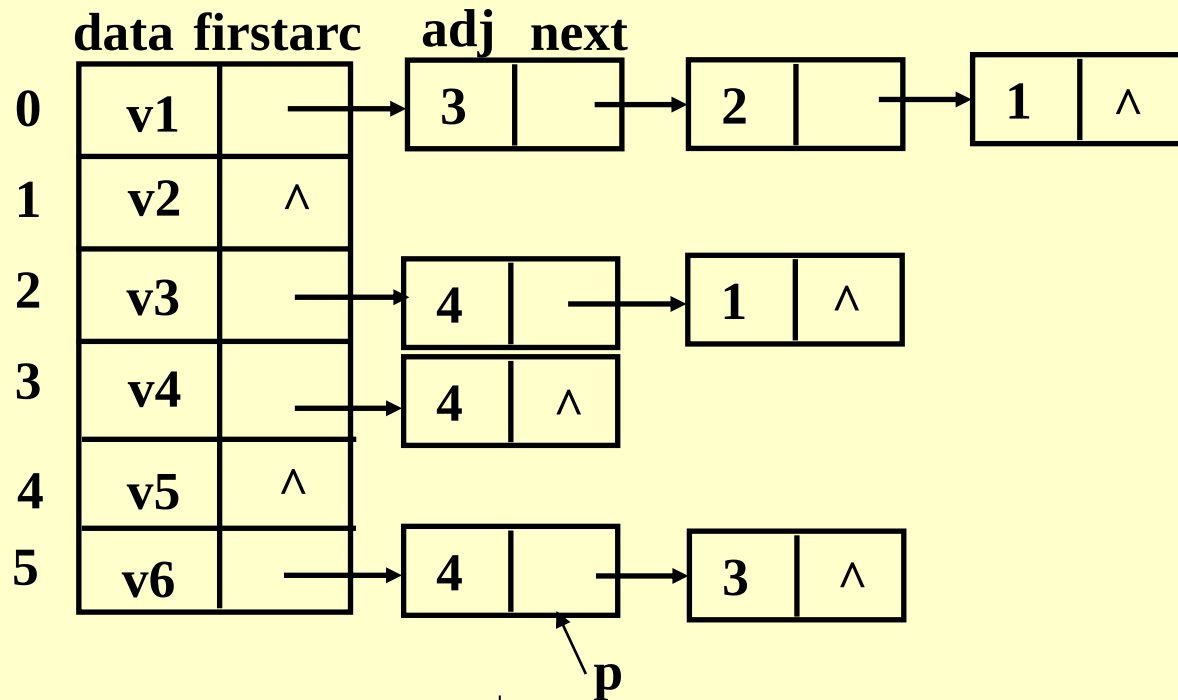




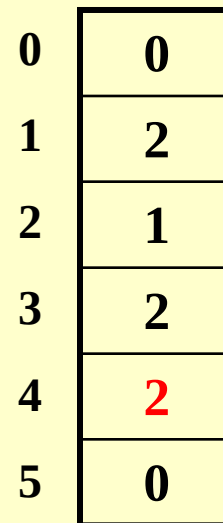
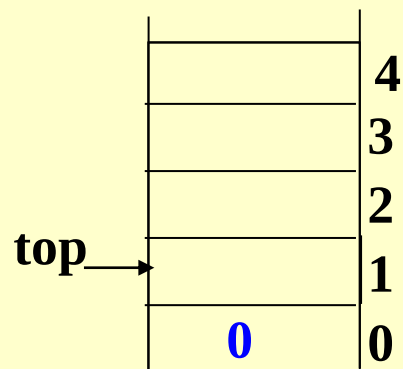
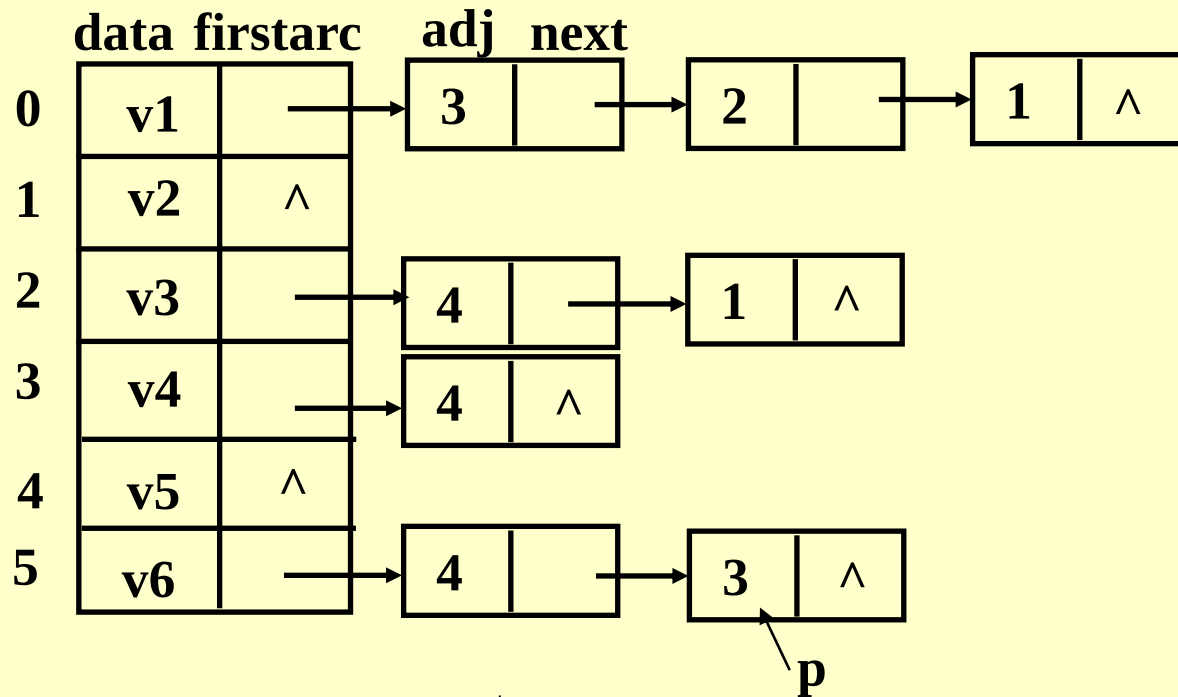
输出序列: **v6**



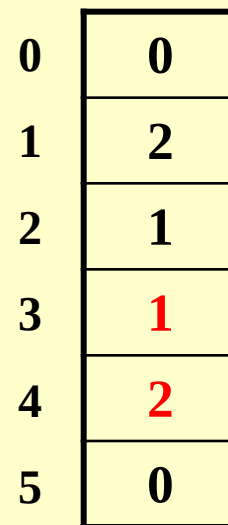
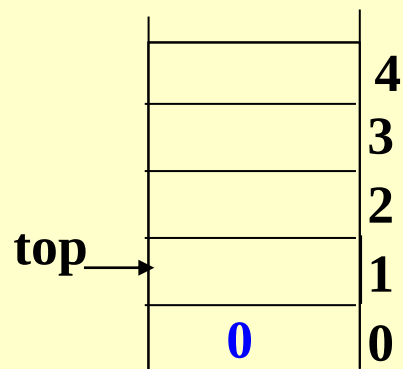
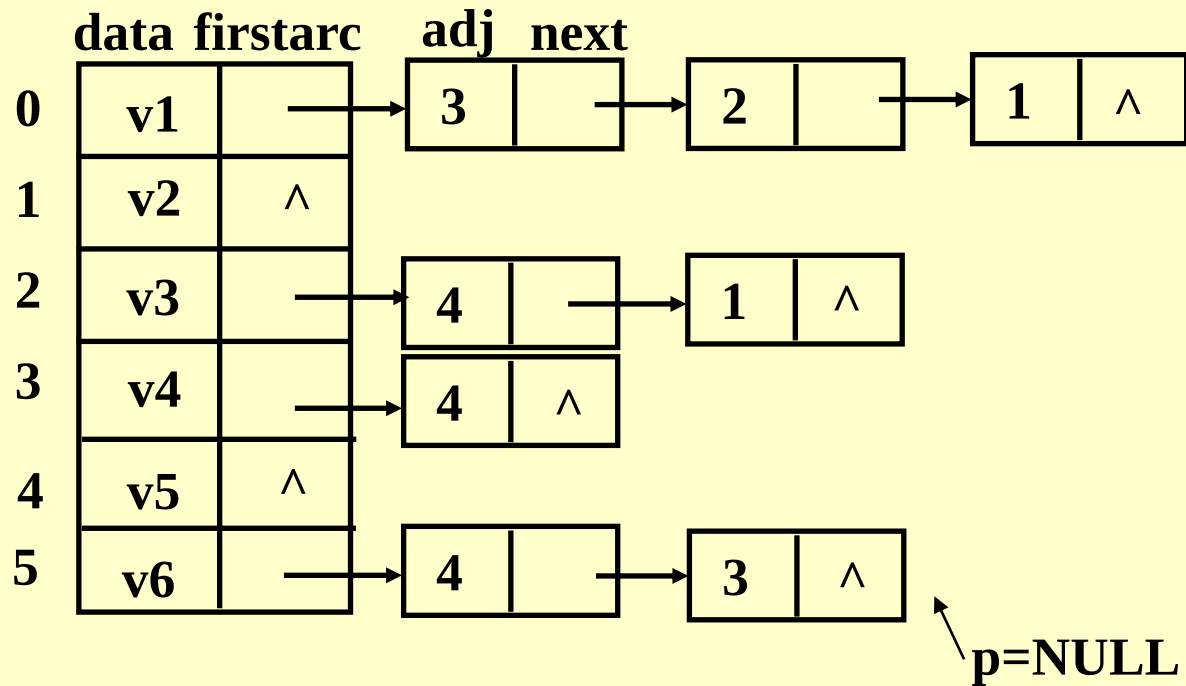
输出序列: **v6**



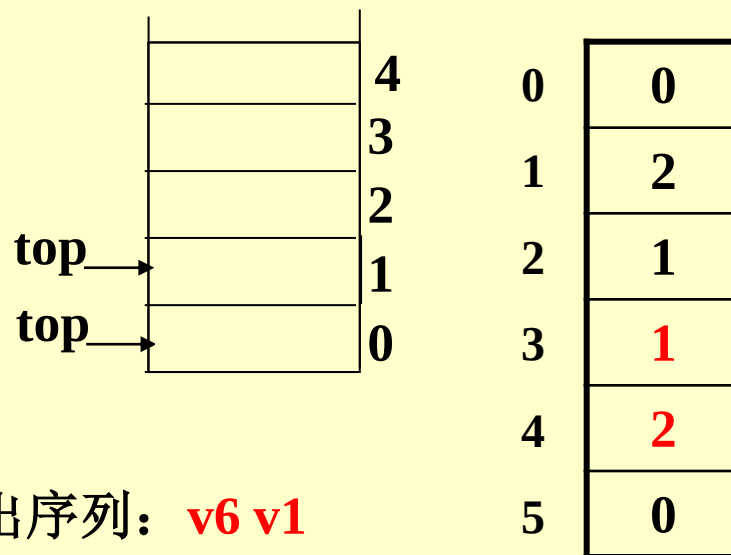
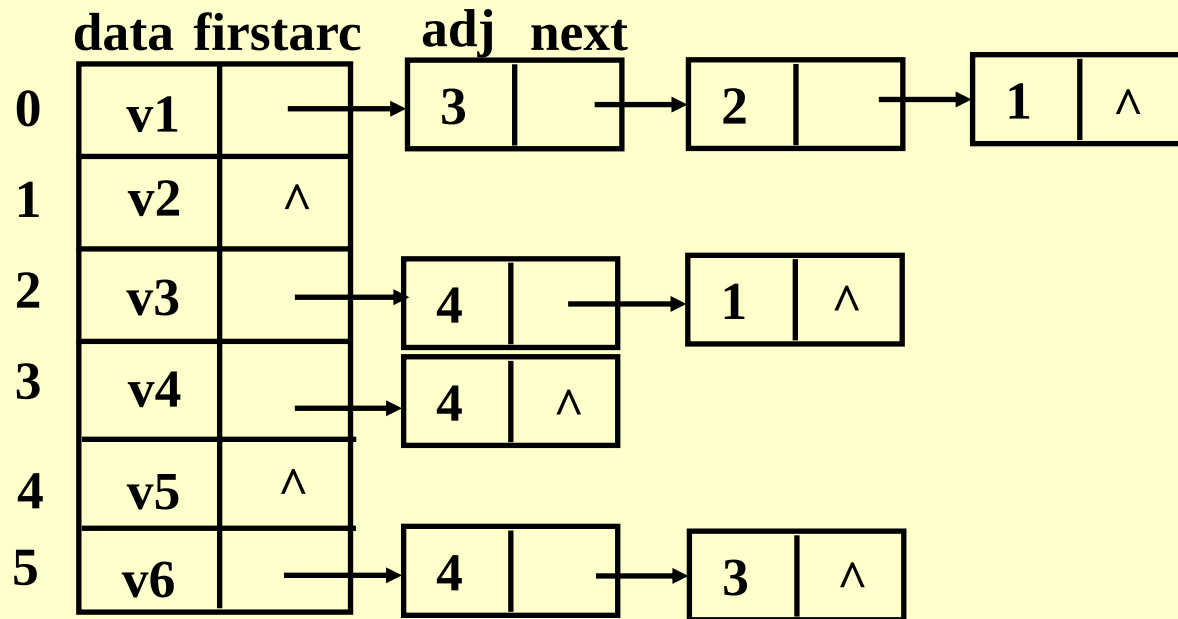
输出序列: **v6**



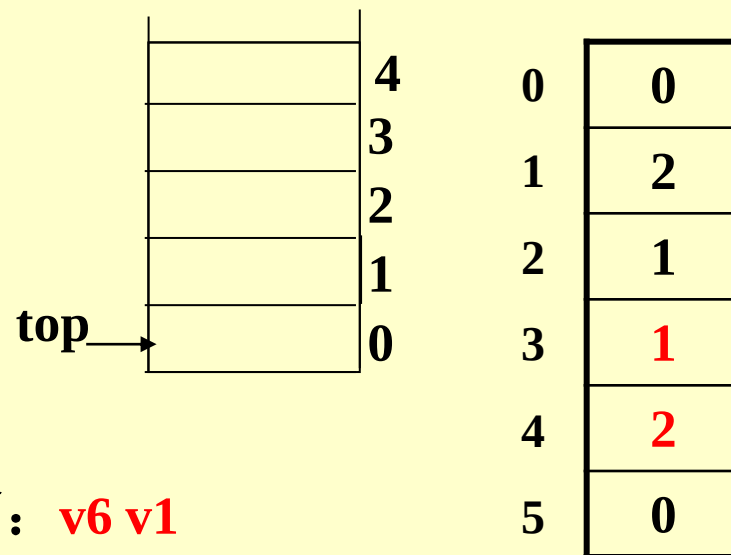
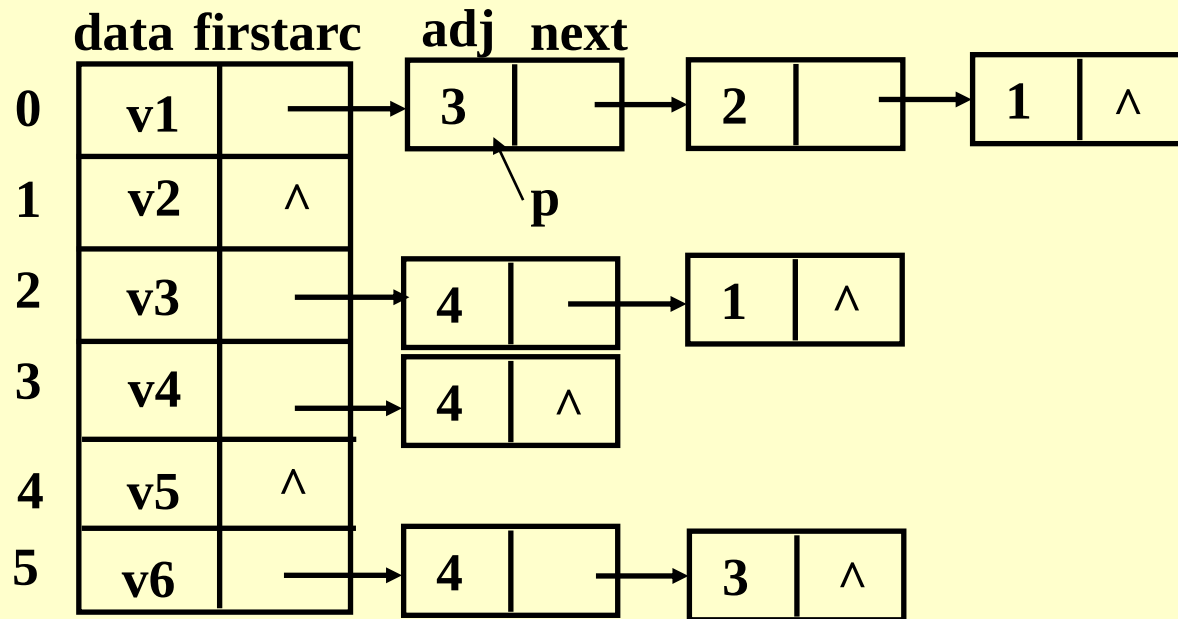
输出序列: **v6**



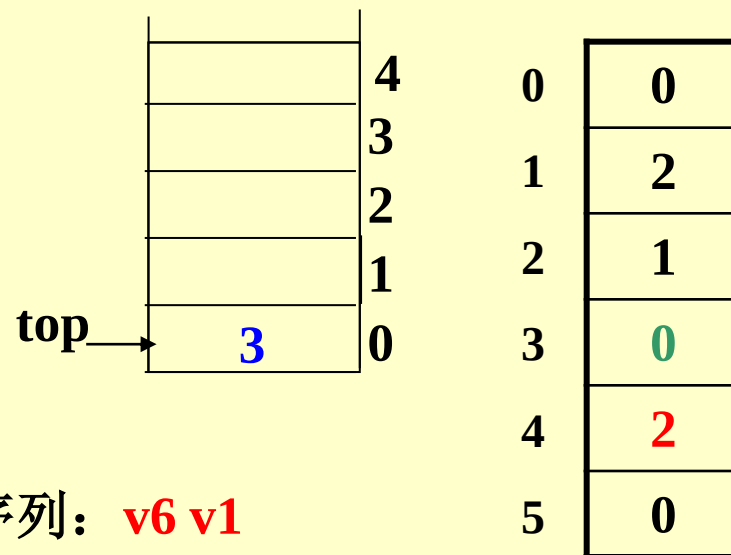
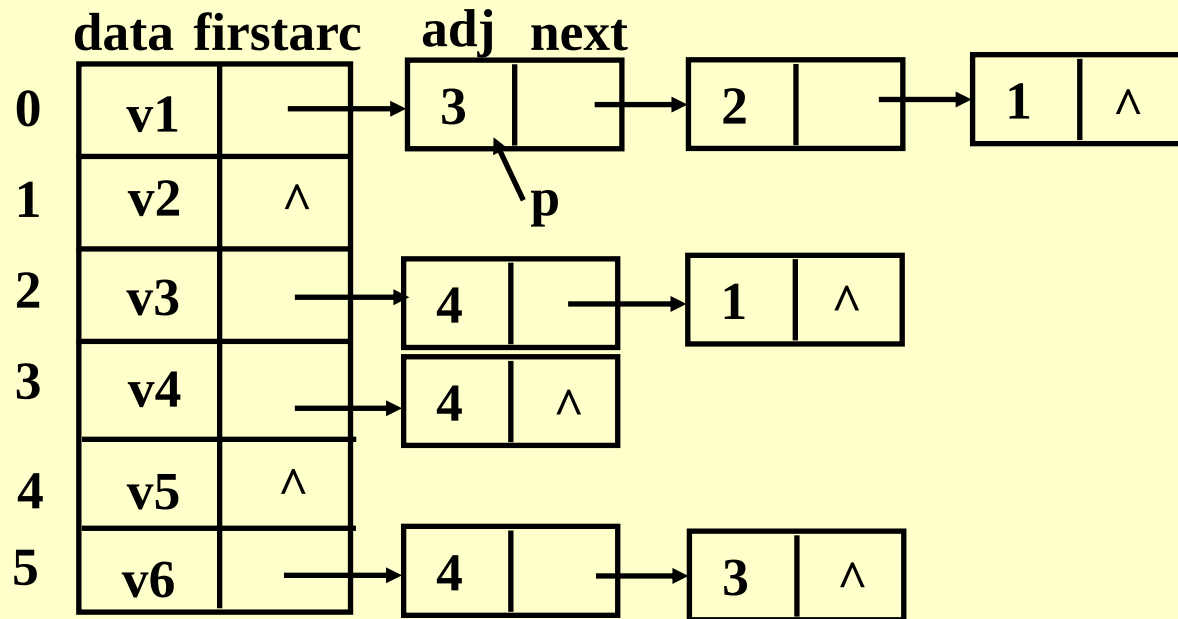
输出序列: **v6**



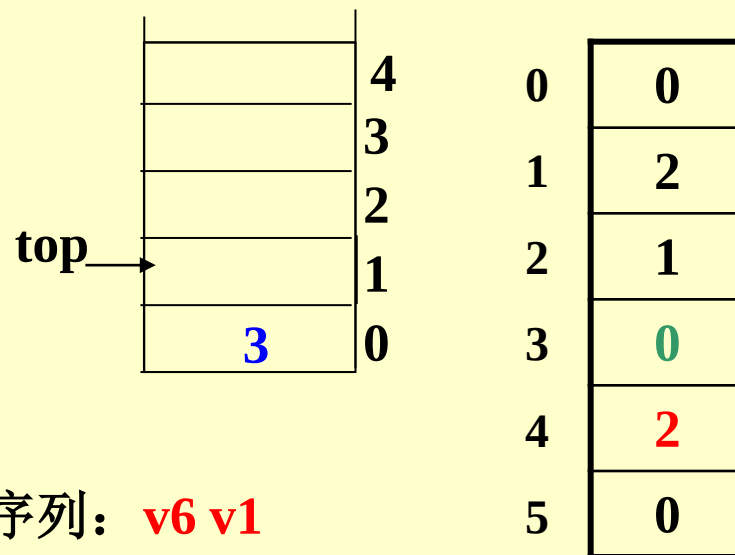
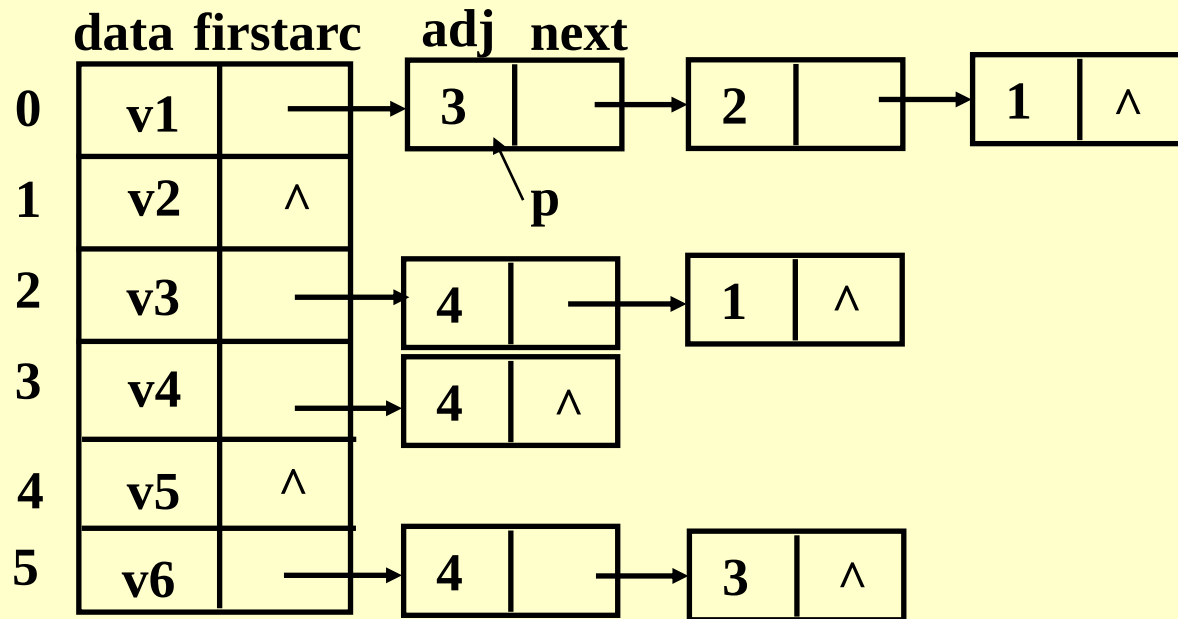
输出序列: **v6 v1**



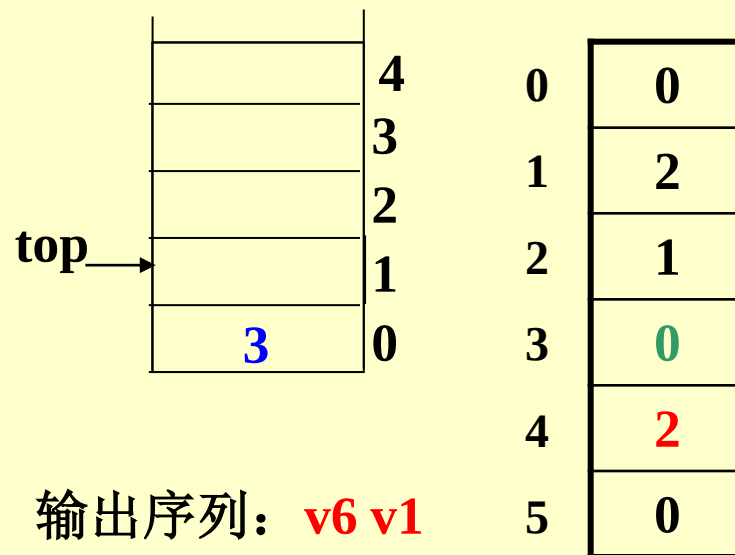
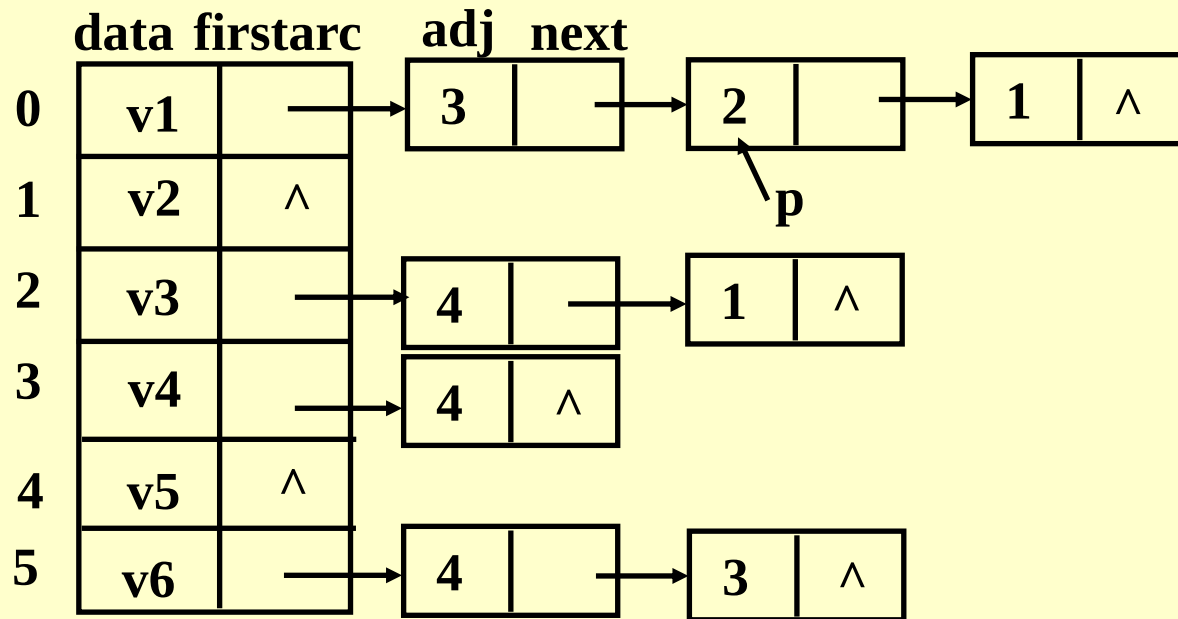
输出序列: **v6 v1**



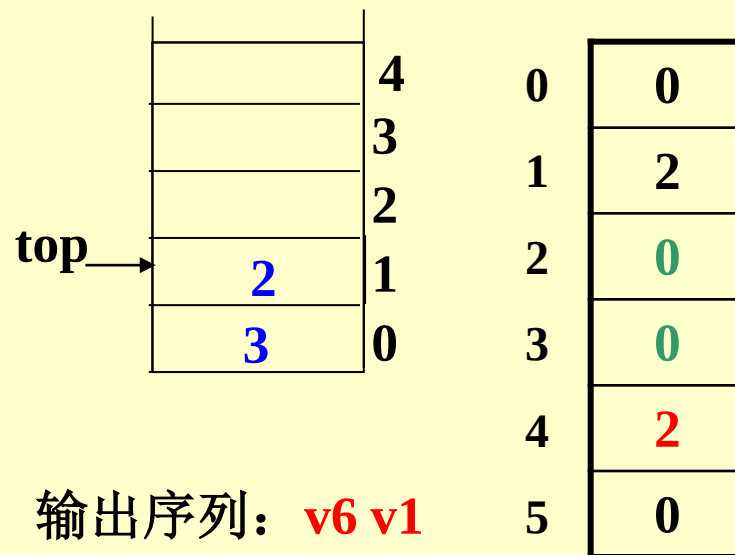
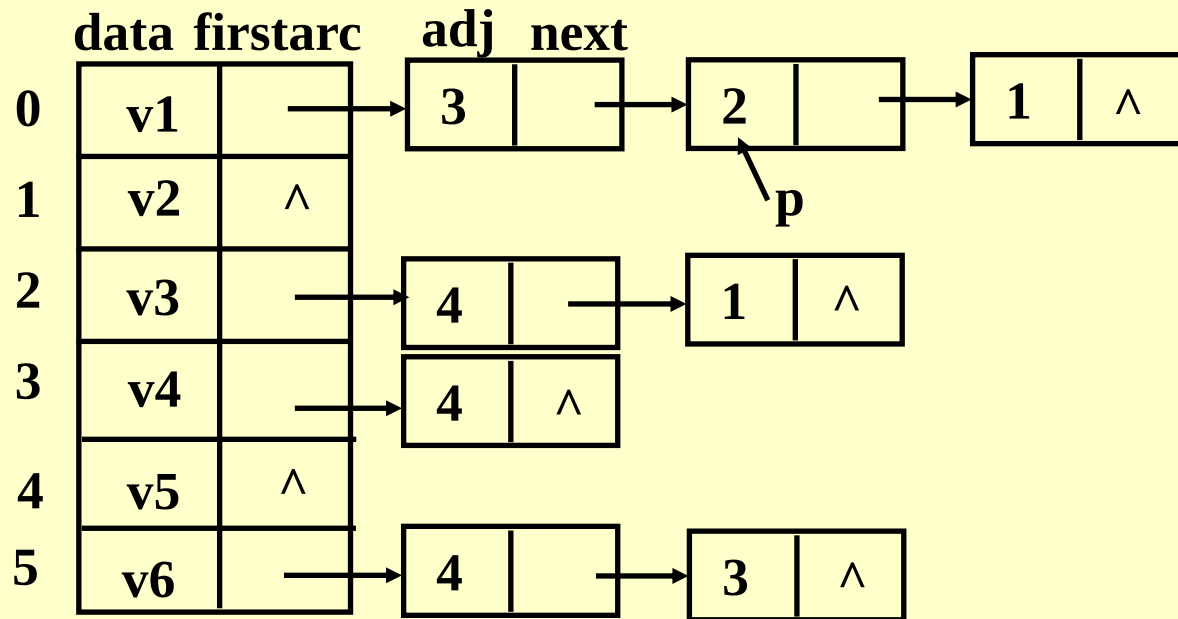
输出序列: **v6 v1**



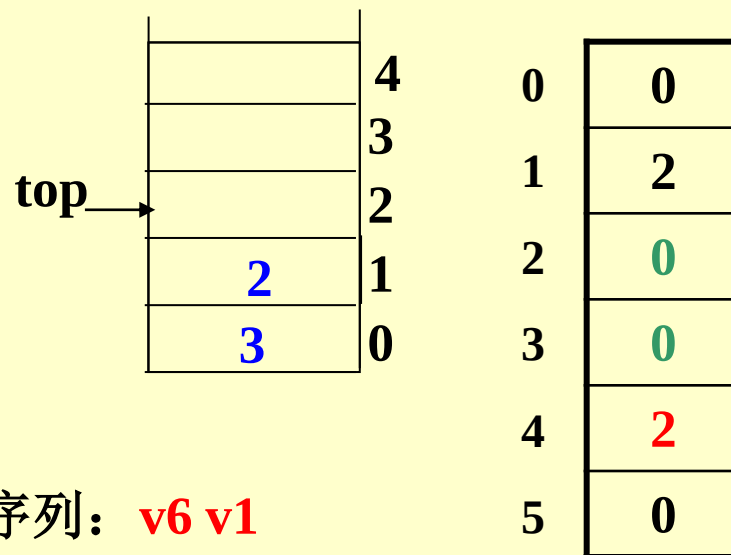
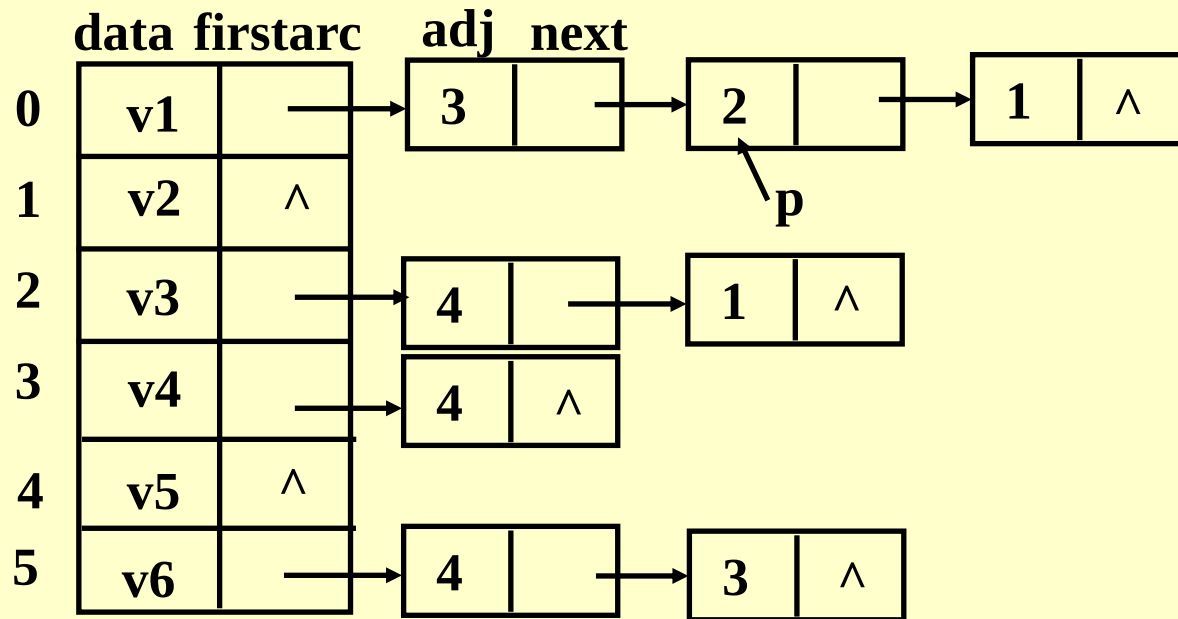
输出序列: **v6 v1**



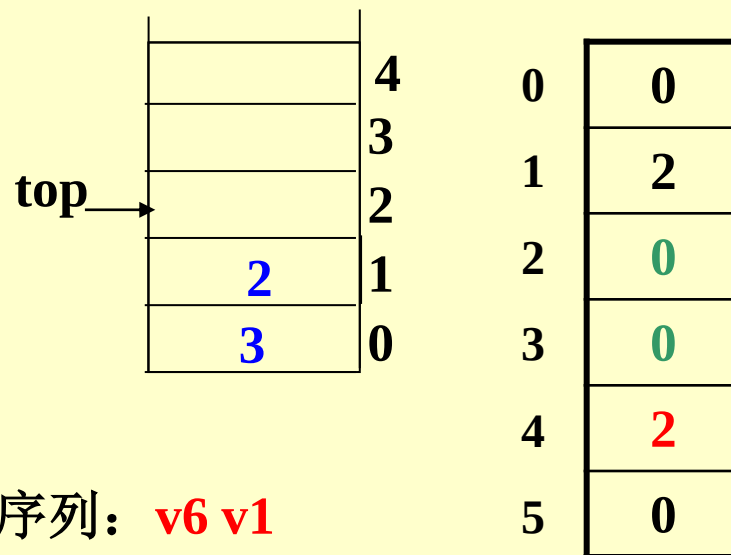
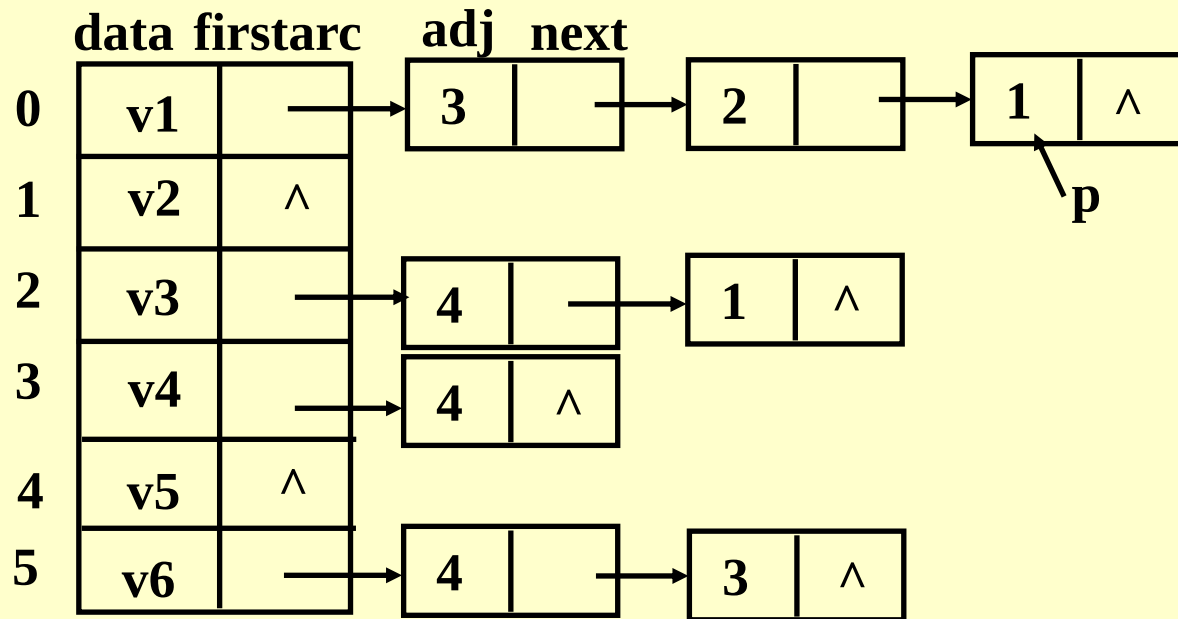
输出序列: **v6 v1**



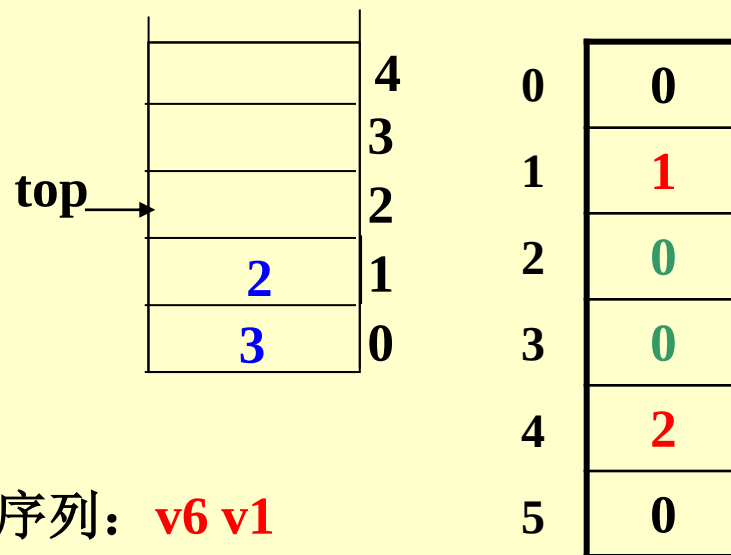
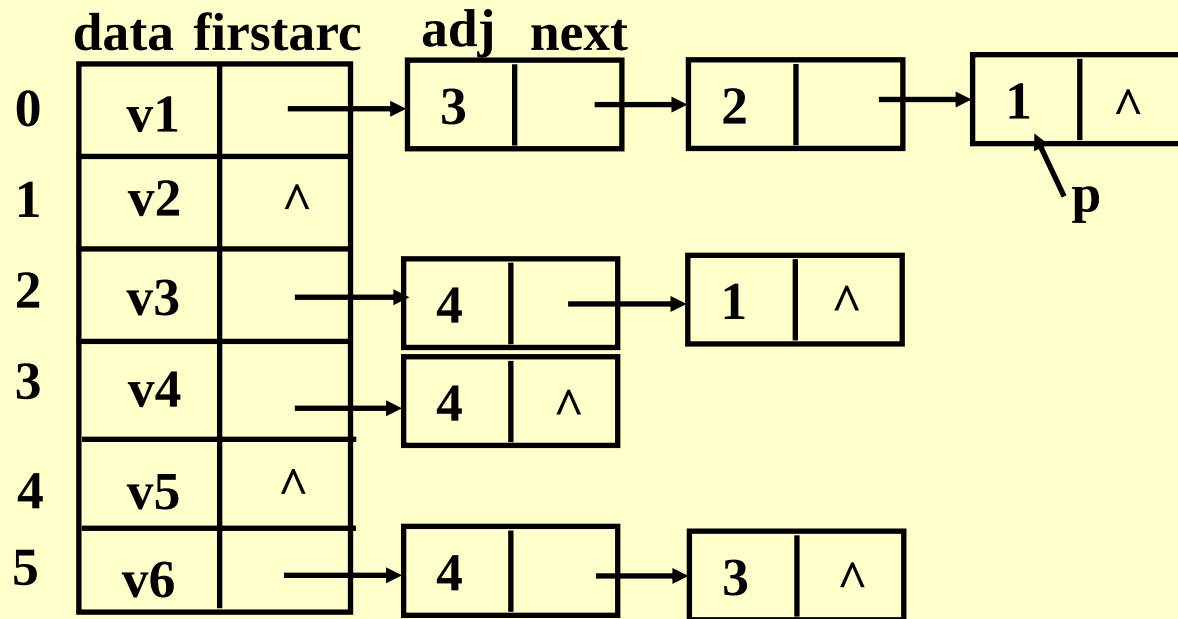
输出序列: **v6 v1**



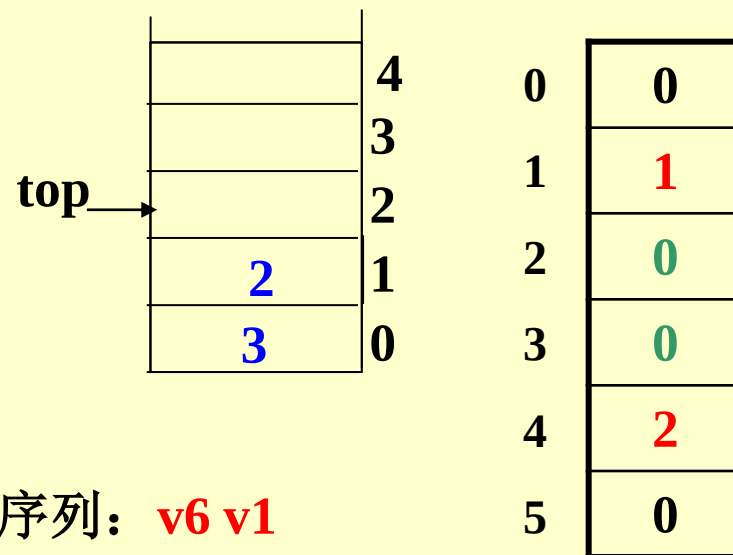
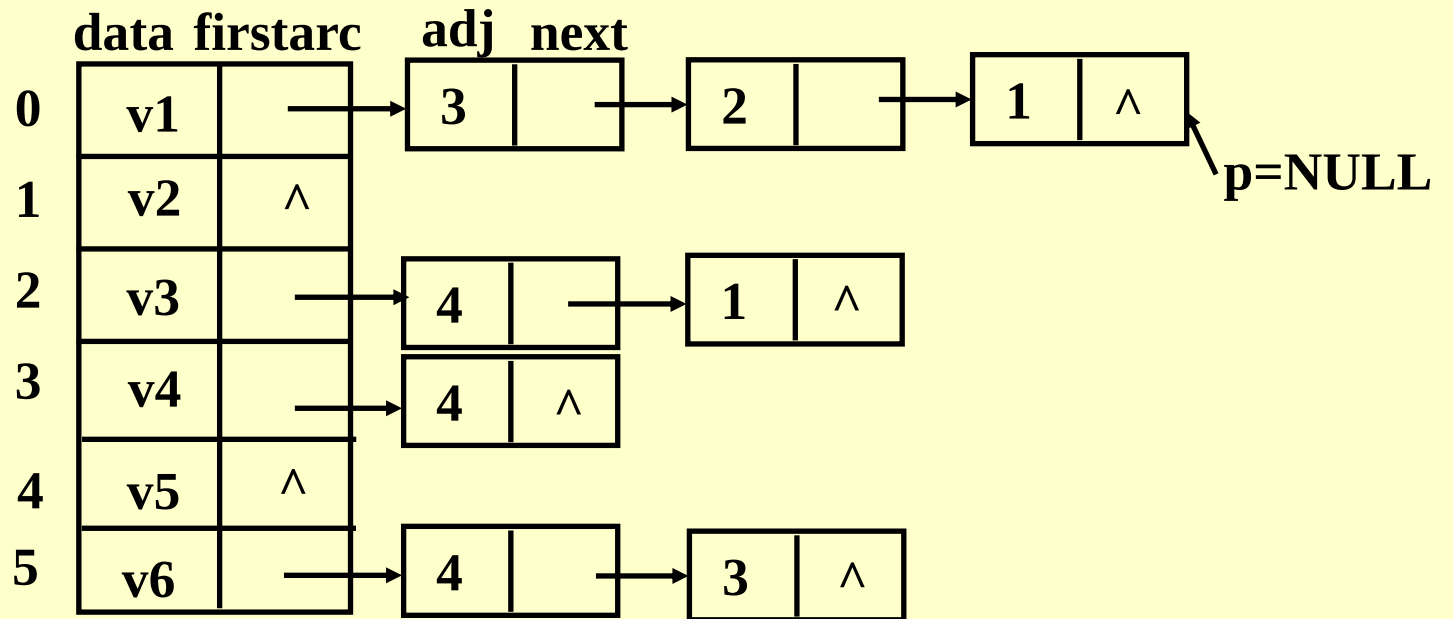
输出序列: **v6 v1**

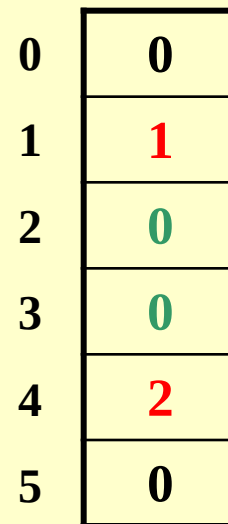
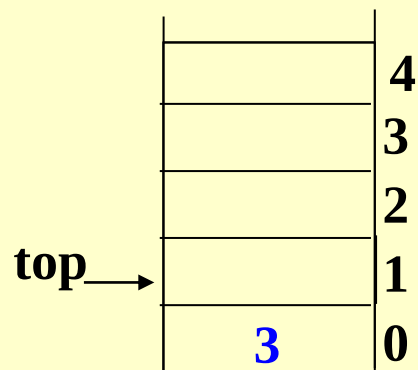
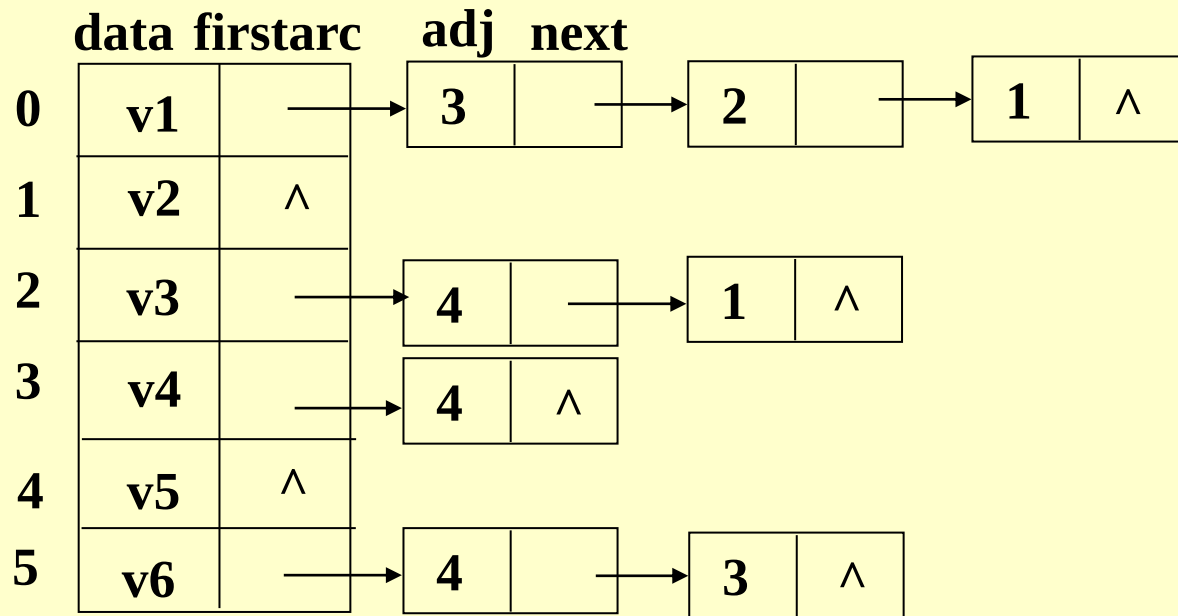


输出序列: **v6 v1**

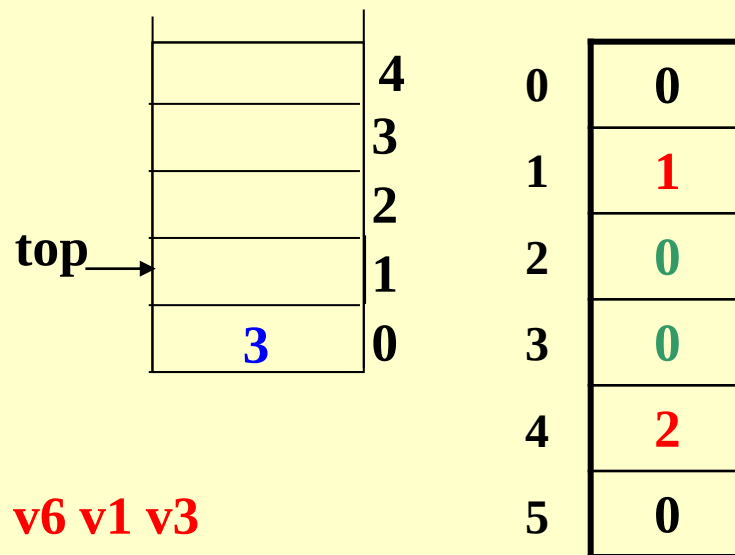
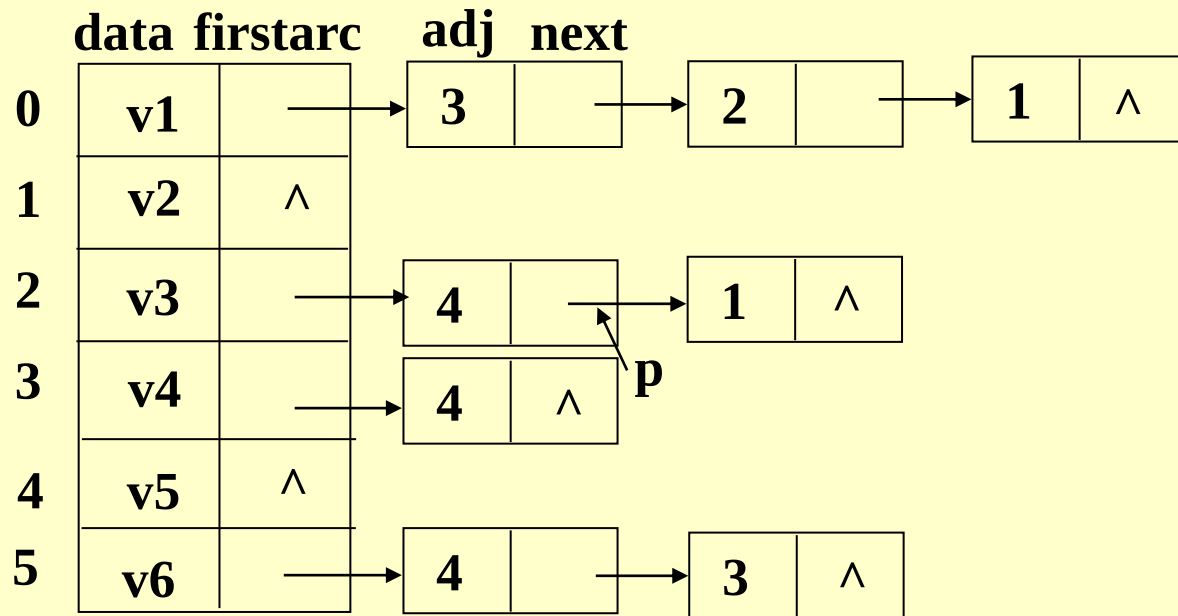


输出序列: **v6 v1**

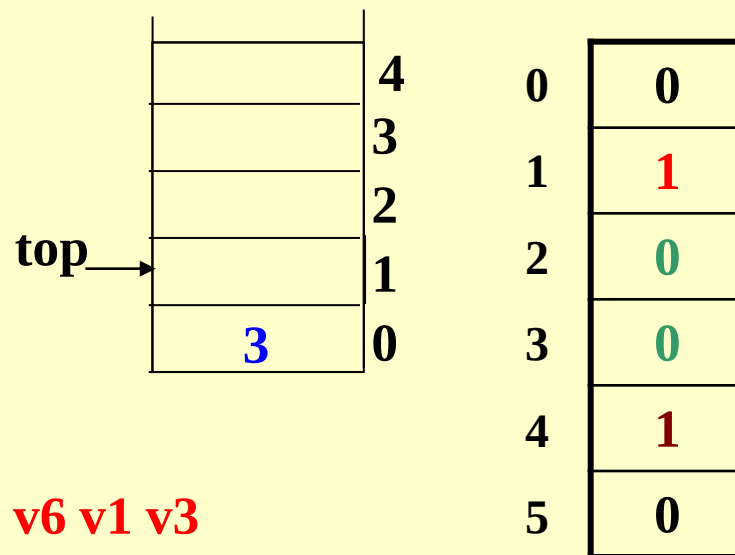
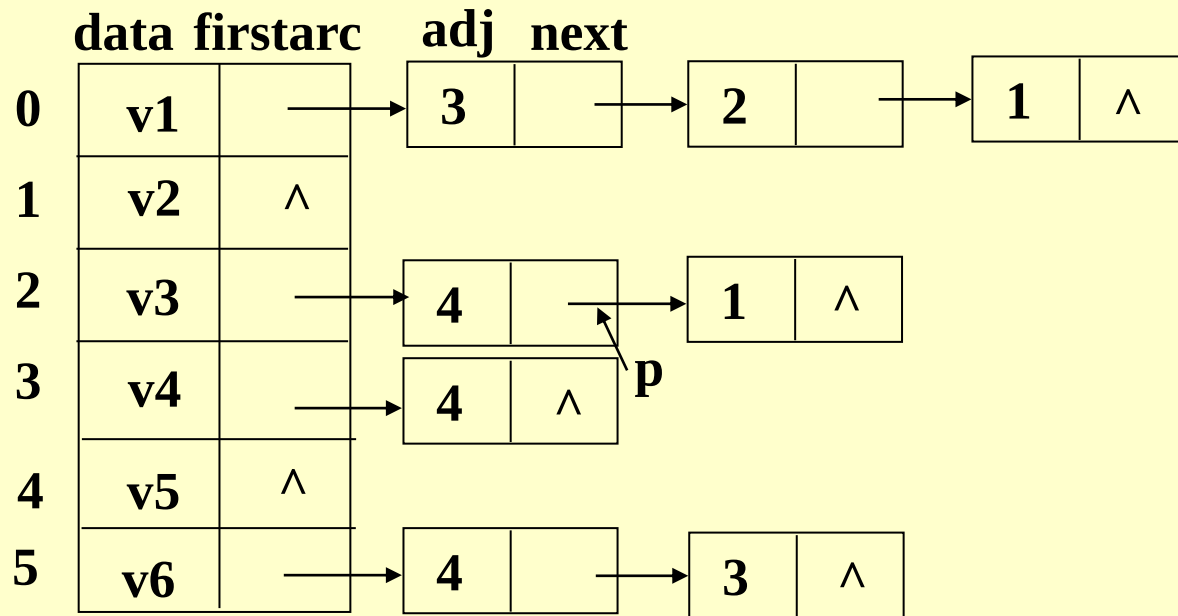




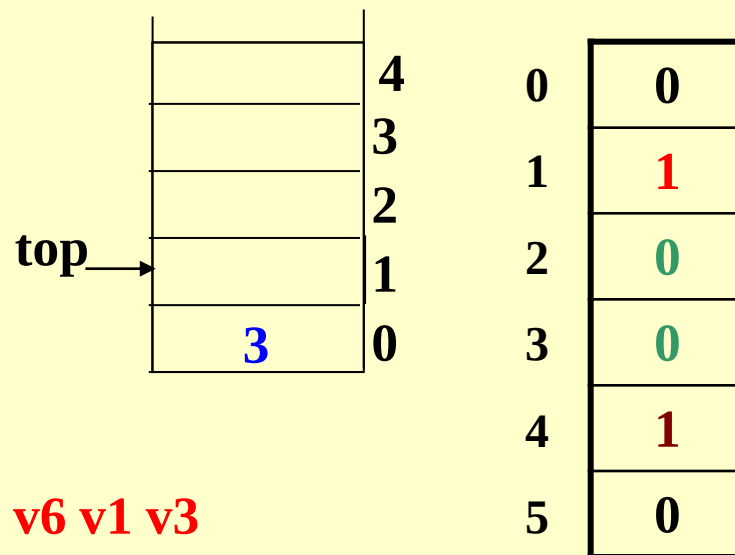
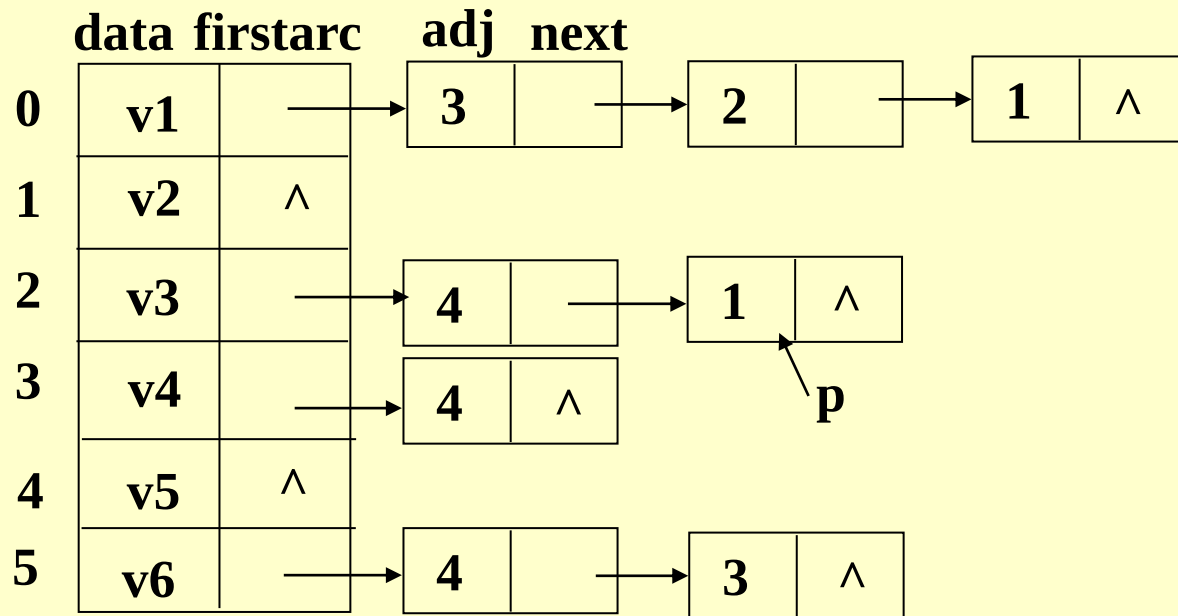
输出序列: **v6 v1 v3**



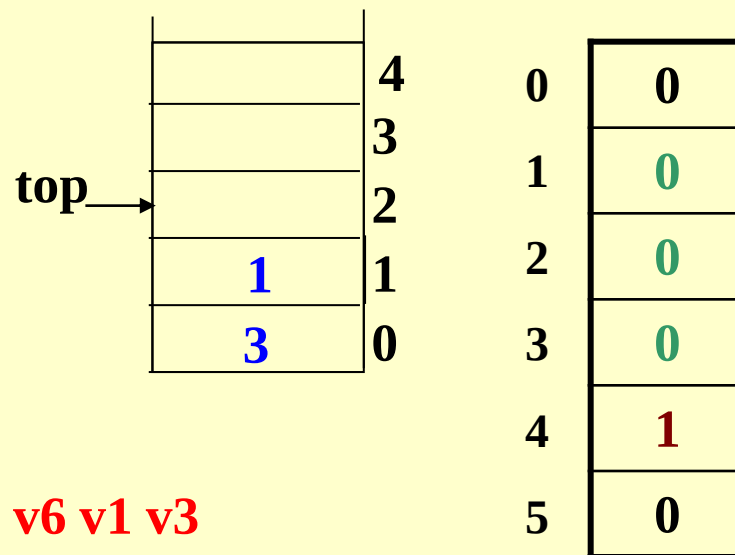
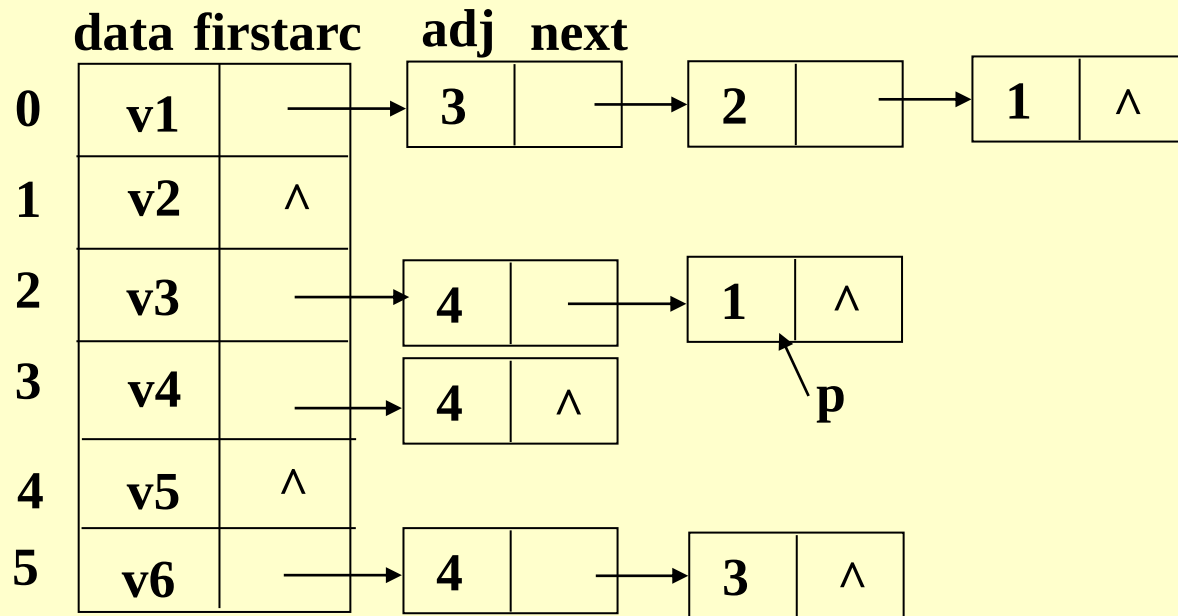
输出序列: **v6 v1 v3**



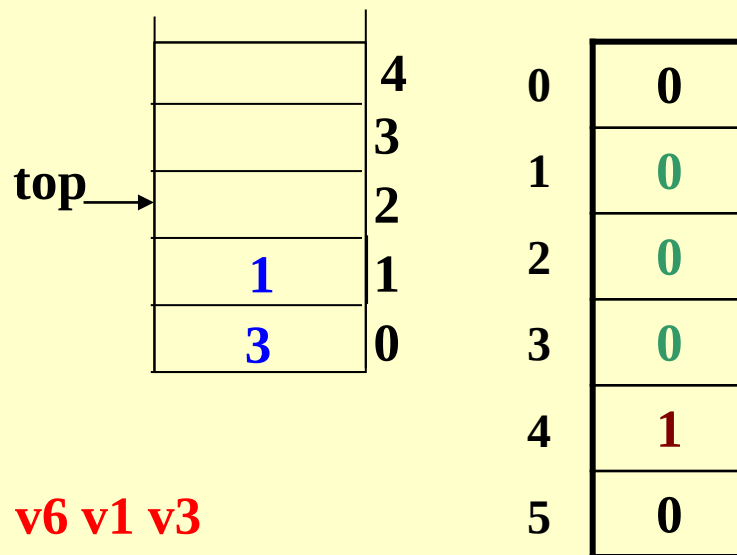
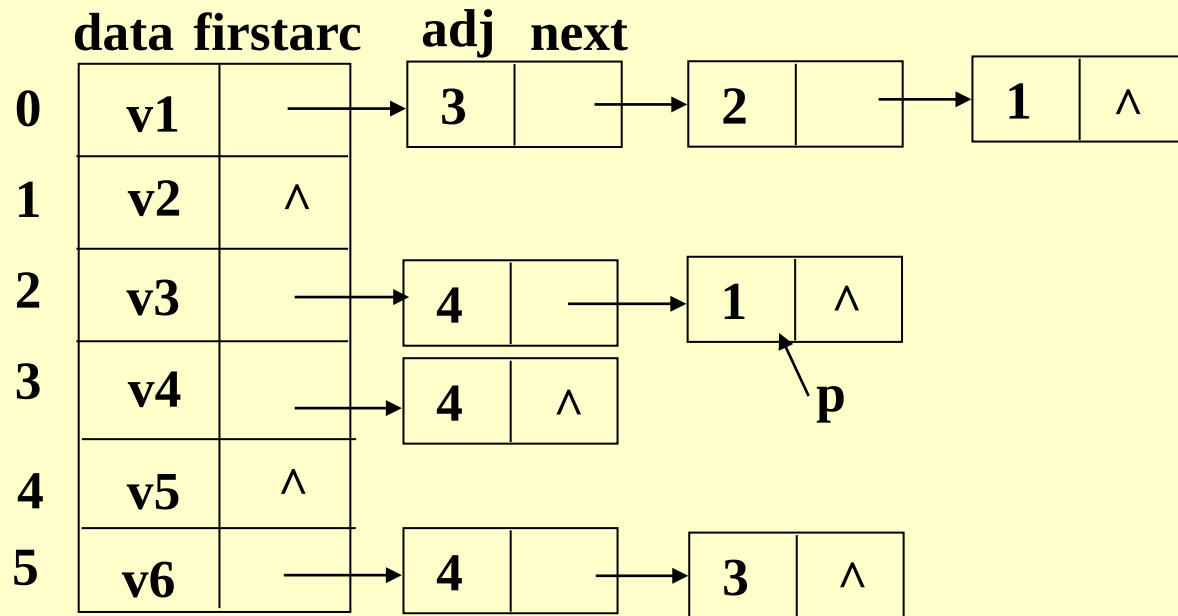
输出序列: **v6 v1 v3**



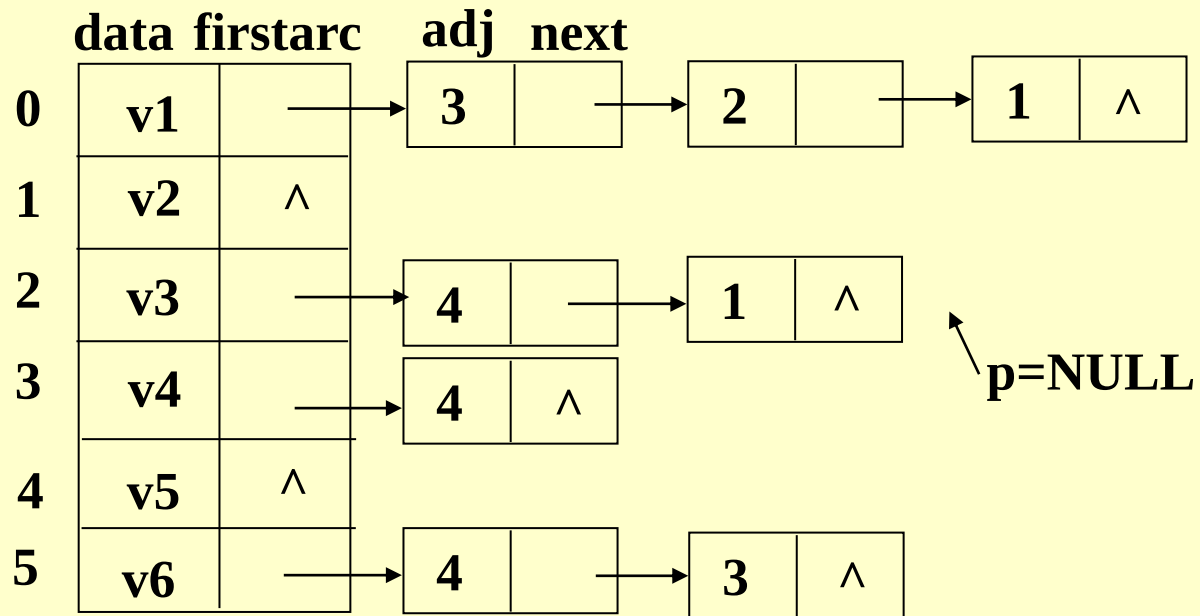
输出序列: **v6 v1 v3**



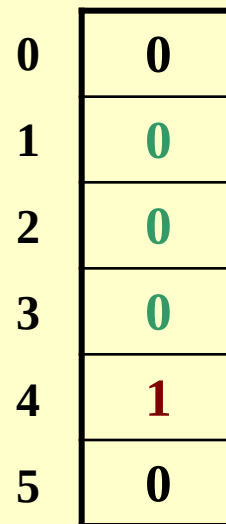
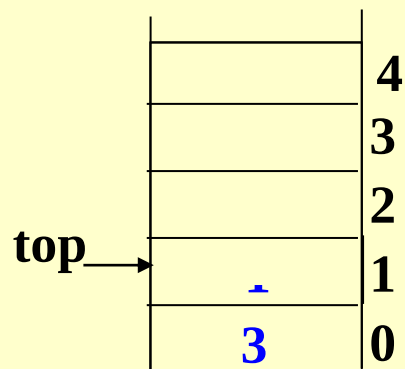
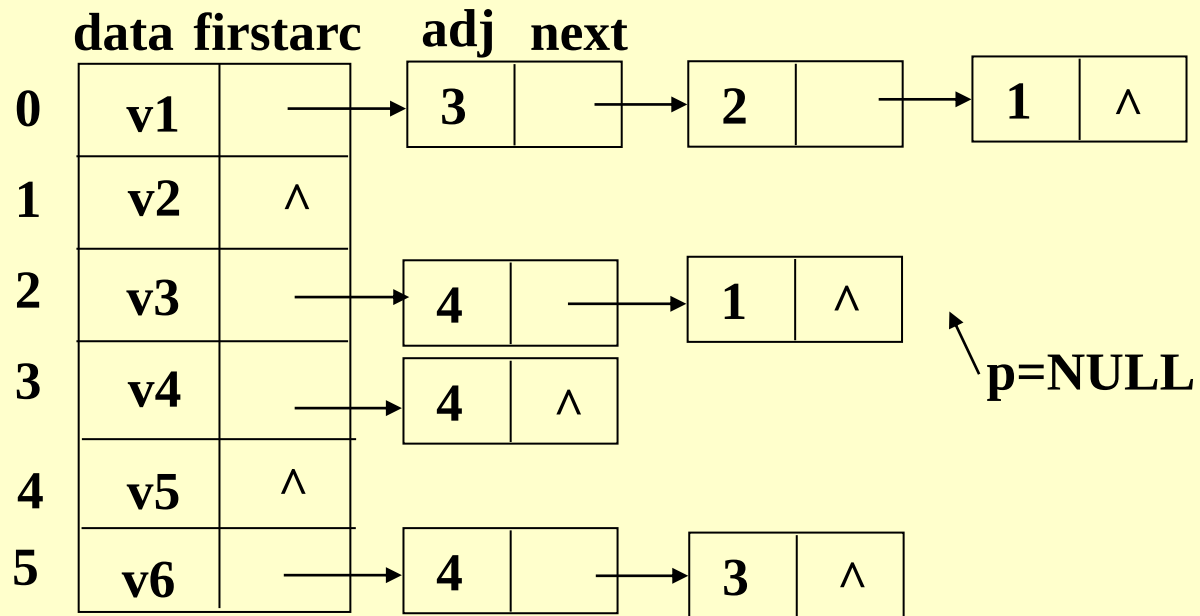
输出序列: **v6 v1 v3**



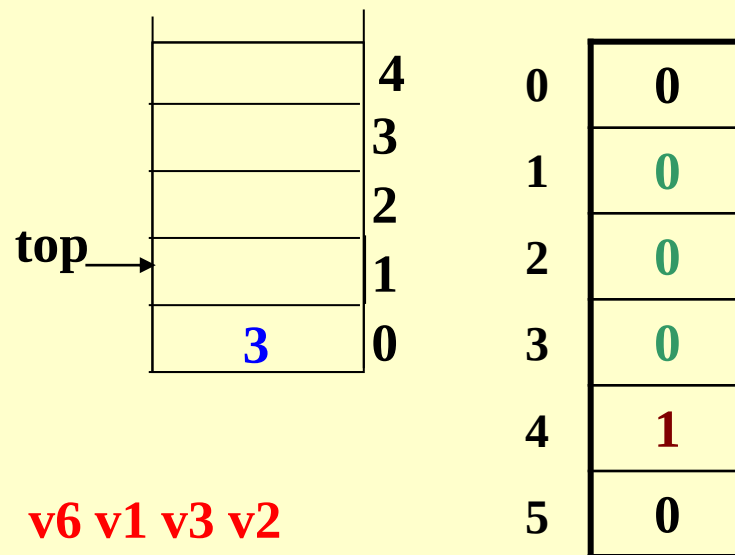
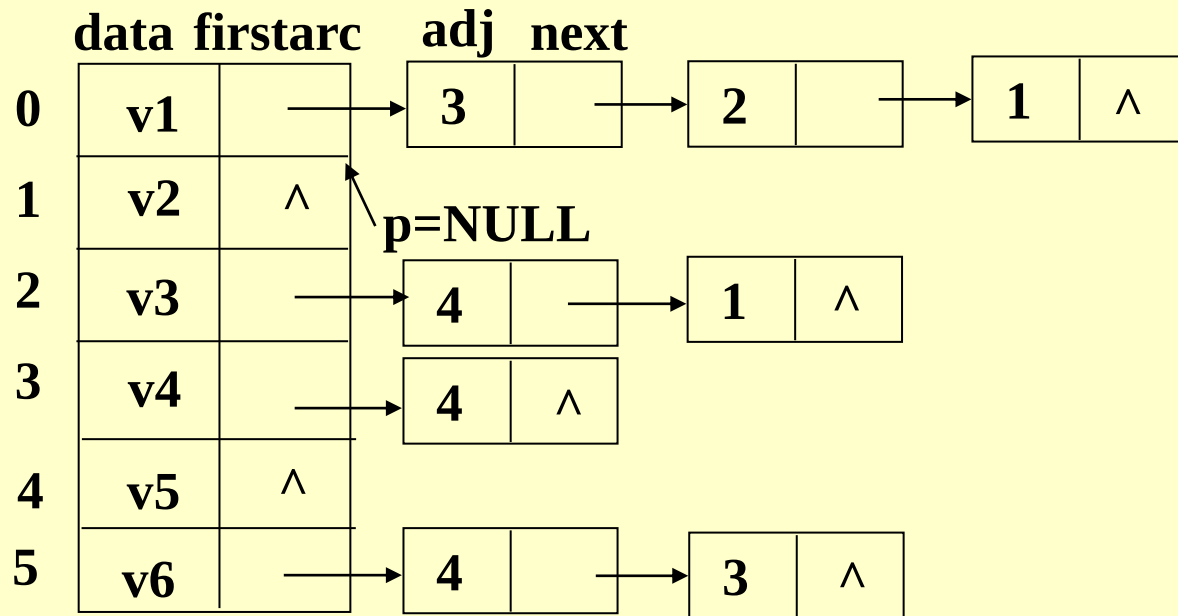
输出序列: **v6 v1 v3**



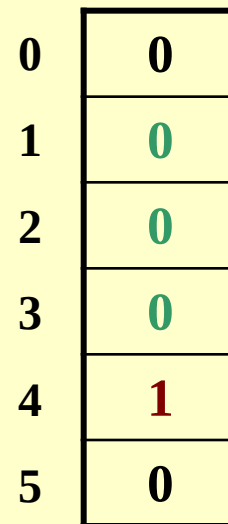
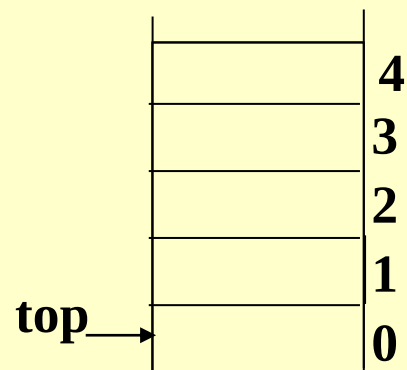
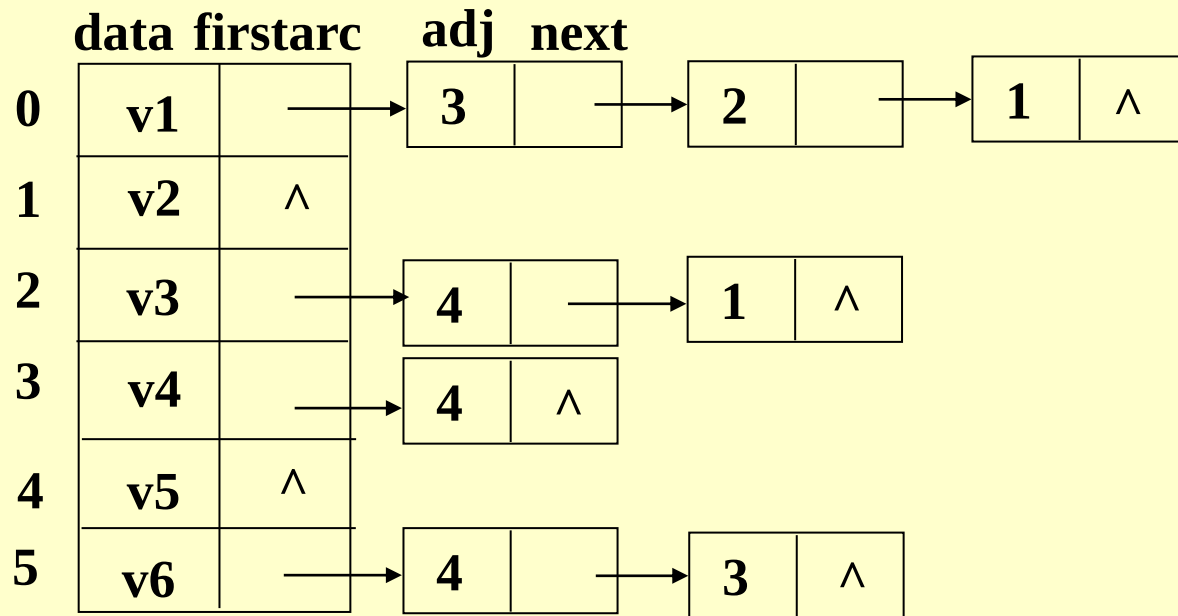
输出序列: **v6 v1 v3**



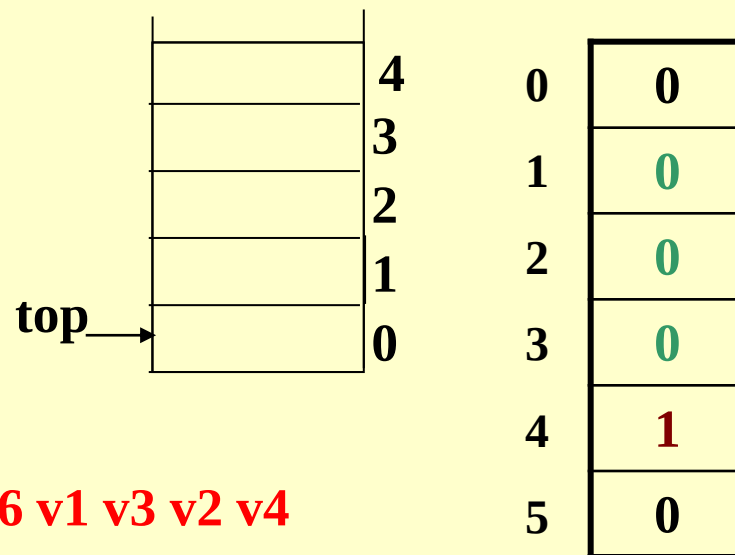
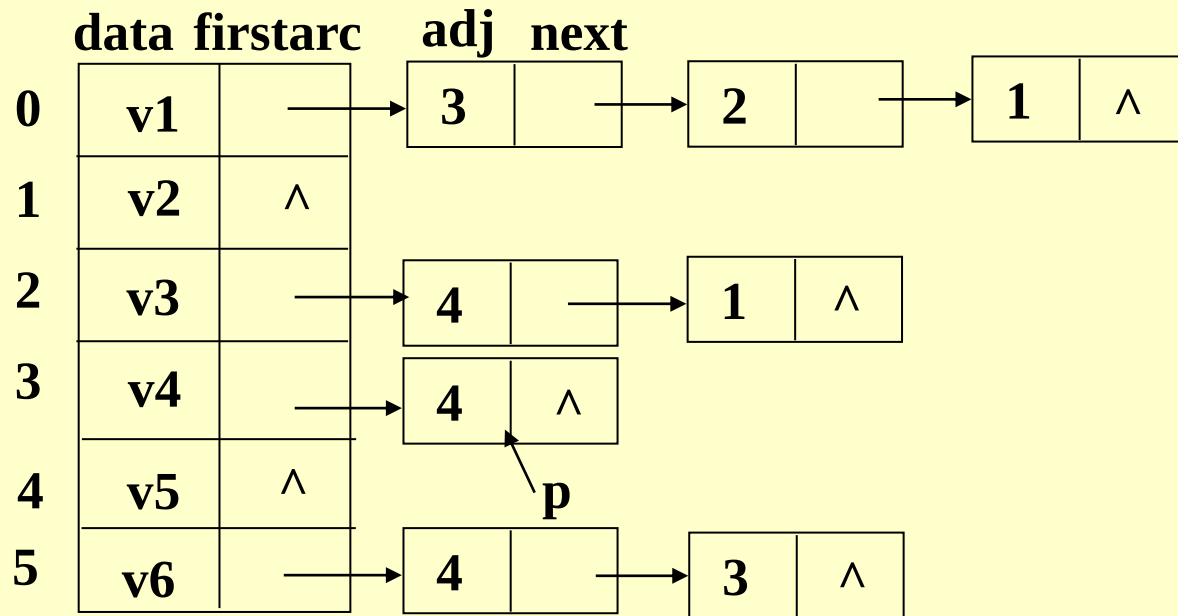
输出序列: **v6 v1 v3 v2**



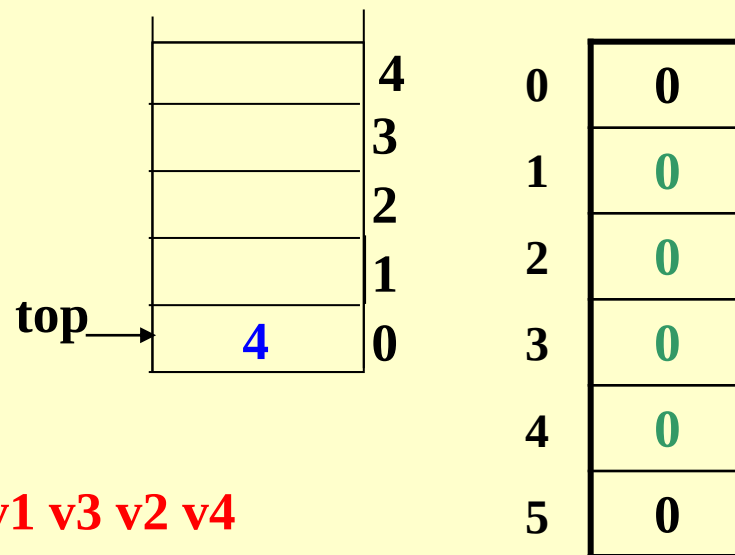
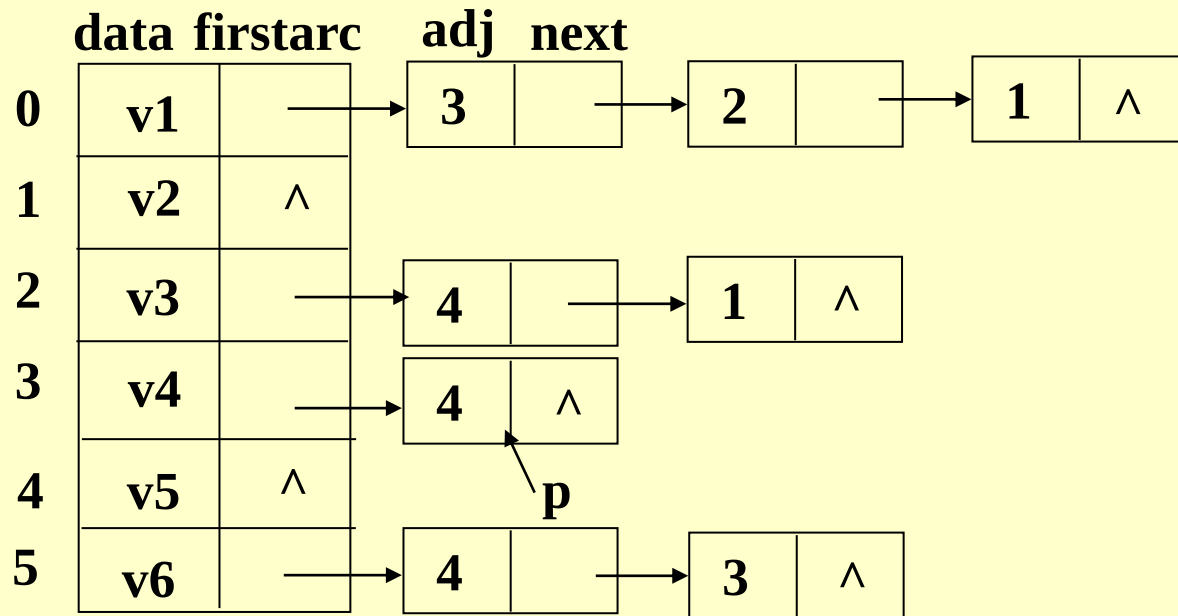
输出序列: **v6 v1 v3 v2**



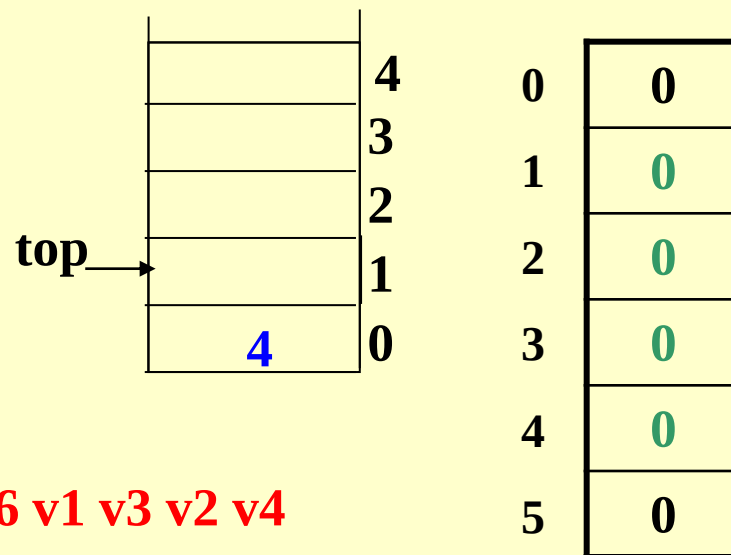
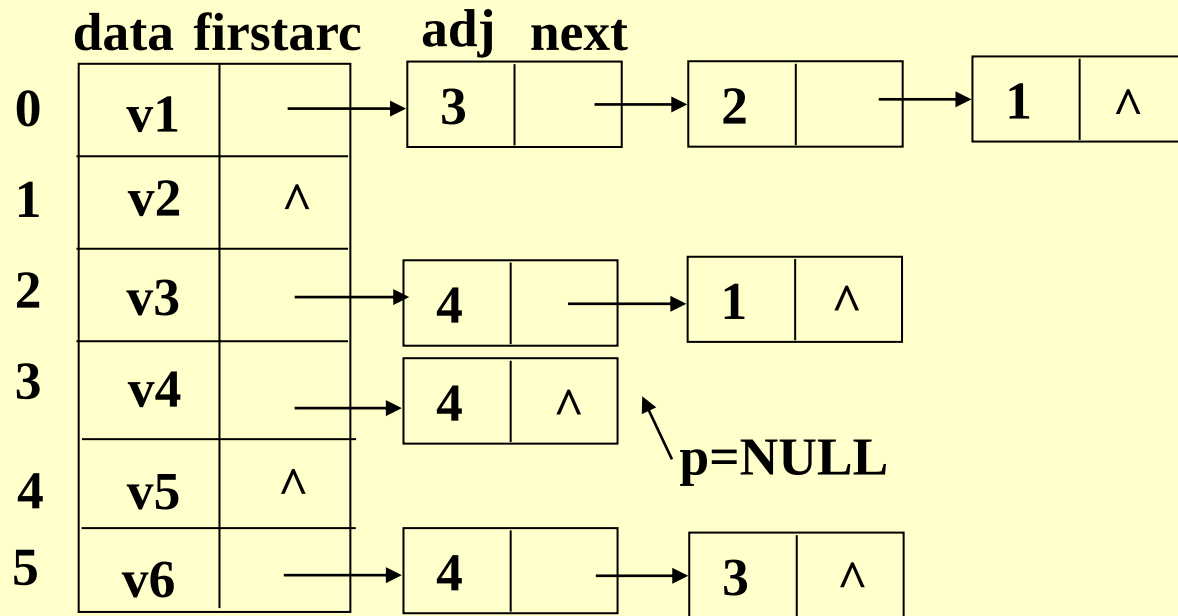
输出序列: **v6 v1 v3 v2 v4**



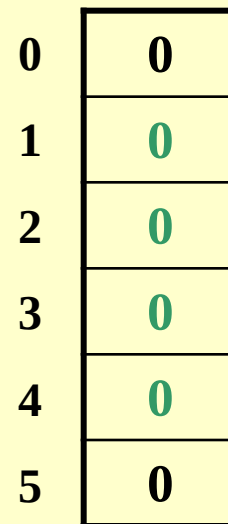
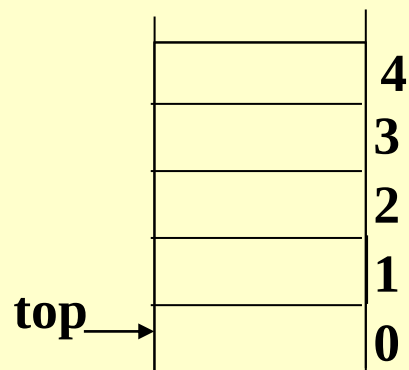
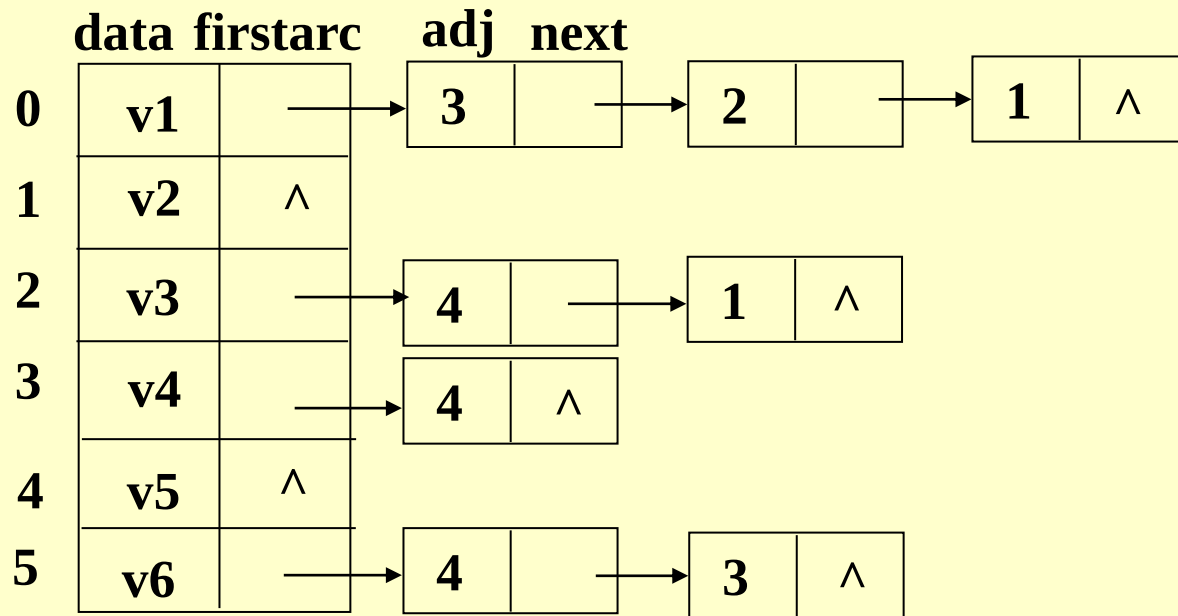
输出序列: **v6 v1 v3 v2 v4**



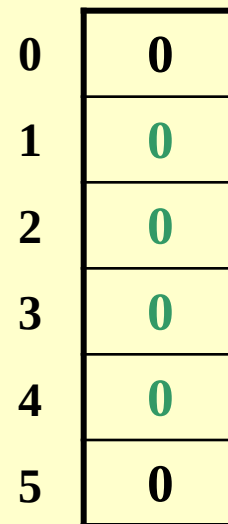
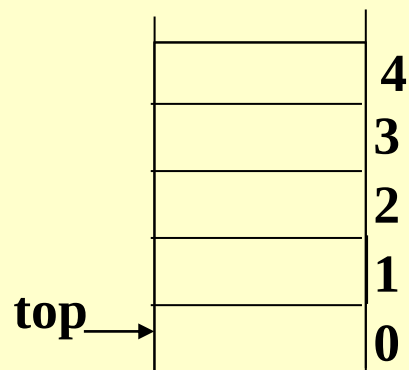
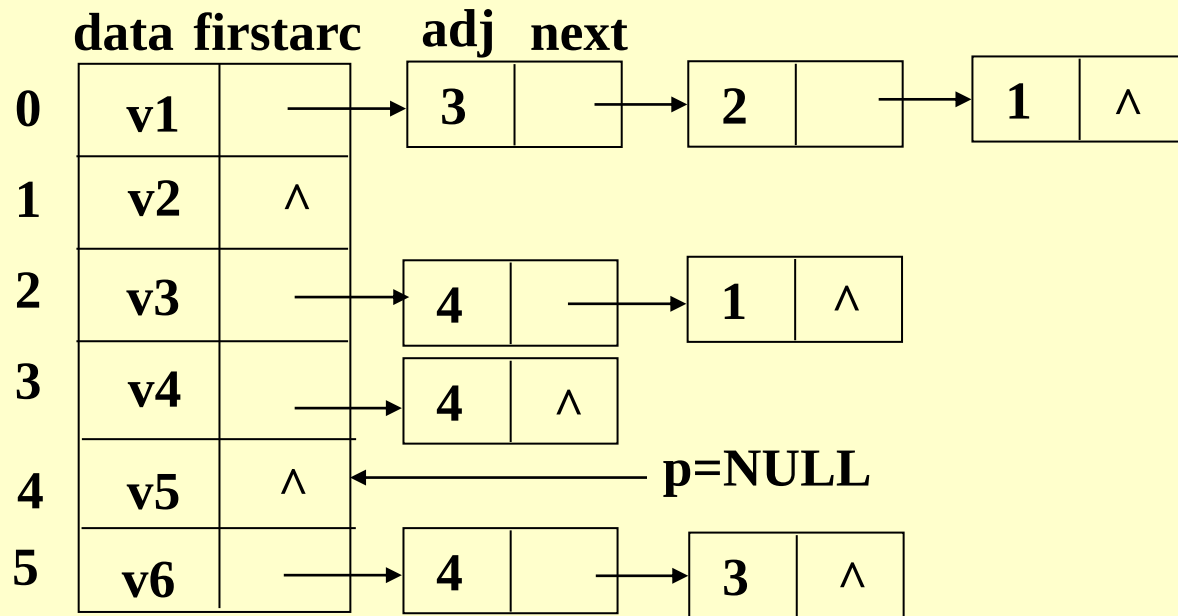
输出序列: **v6 v1 v3 v2 v4**



输出序列: **v6 v1 v3 v2 v4**



输出序列: **v6 v1 v3 v2 v4 v5**



输出序列: **v6 v1 v3 v2 v4 v5**

算法分析

- 建各个顶点入度的时间: $O(e)$;
- 建立0入度顶点栈的时间: $O(n)$;
- 拓扑排序: 每个顶点进栈一次, 出栈一次, 入度减1共执行 e 次, 因此总时间复杂度为 $O(n+e)$.

邻接表实现

```
#define MAX_NUM 20//最大顶点数目  
#define MAX_NUMA 40//最大弧数目
```

```
typedef enum {DG,DN,UDG,UDN} GraphKind;//图类型
```

```
typedef struct Acnode{  
    int adjvex;//存放顶点在数组中的位置  
    int value;//权值，这里定义为整型  
    struct Acnode *next;  
    char *info;//存放弧的信息  
}Acnode;//弧的定义
```

```
typedef struct {  
    char data;  
    Acnode *firstarc;  
}VertexNode;//顶点定义
```

```
typedef struct{  
    VertexNode Vertex[MAX_NUM];  
    int Vexnum,Arcnum;//当前顶点数目、弧数目  
    GraphKind kind;//图的类型  
}ALGraph;//图的定义
```

创建图

```
void CreatGraph_DN(ALGraph *G)
{  Acnode *p, *q; int i, j, sign1, sign2, Value; VertexNode v1, v2;//弧头弧尾
  G->kind = DN;
  printf("请输入该有向网的顶点数目和弧的数目: ");
  scanf("%d%d", &G->Vexnum, &G->Arcnum);
  printf("请初始化%d个顶点, 空格或回车分隔: ", G->Vexnum);
  for (i = 0; i<G->Vexnum; i++) {
    scanf(" %c", _____);
    _____ = NULL; } //头结点第一个邻接点赋空
  printf("请初始化顶点之间的关系 (%d条弧): \n", G->Arcnum);
  printf("形式: 顶点1 顶点2 权值 (表示顶点1邻接到顶点2), 如a b 1,回车下一条弧\n");
  fflush(stdin);
  for (i = 0; i<G->Arcnum; i++) { //插入每个弧
    sign1 = -1; sign2 = -1;
    fflush(stdin);
    scanf("%c %c %d", _____, _____, _____);
    for (j = 0; j<G->Vexnum; j++) { //定位输入顶点在顶点数组中的位置
      if (G->Vertex[j].data == v1.data)      sign1 = j;
      if (G->Vertex[j].data == v2.data)      sign2 = j;}
    if (sign1 == -1 || sign2 == -1) printf("信息输入有误! \n");
    else {  p = (Acnode*)malloc(sizeof(Acnode)); //申请空间
      p->next = NULL;
      p->adjvex = sign2;
      p->value = Value;//构造弧表结点
      q = G->Vertex[sign1]._____ ;
      if (q == NULL)//链入弧表结点至顶点链表中
        G->Vertex[sign1].firstarc = _____ ;
      else{  while (q->next != NULL) //找到最后一个结点
        q = q->next;
        q->next = _____ ;}
    }
  }
}
```

邻接表实现

```
void TopologicalSort1(ALGraph G)
{
    int indegree[MAX_NUM], count = 0, k, i;
    Acnode *p;
    Acnode e;
    Stack s;    s = (Stack)malloc(sizeof(SNode));    s->next = NULL;//初始化链栈
    for (i = 0; i < G.Vexnum; i++)
        _____ = 0;//初始化顶点入度信息
    for (i = 0; i < G.Vexnum; i++)//遍历邻接表求得图中每个顶点的入度
    {
        p = G.Vertex[i]._____ ;
        while (p)
        {    indegree[_____]++;        p = _____ ;}
    }
    for (i = 0; i < G.Vexnum; i++)//找到indegree数组中入度为0的顶点然后入栈
        if (!indegree[i]){ e.adjvex = i;    push(s, _____); }
    while (!IsEmpty(s))
    {
        e.adjvex = pop(s);
        printf(" %c ", G.Vertex[_____ ]);
        count++;//对输出的顶点计数
        for (p = G.Vertex[e.adjvex]._____ ; p != NULL; p = _____ )//用p遍历每个表结点
        {
            k = _____;
            indegree[k]--;
            if (!indegree[k]) { e.adjvex = k;    push(____, _____); }
            //删除下标为p->adjvex的顶点相关的边(入度减1)，并将度为0的顶点入栈
        }
    }
    if (count < G.Vexnum)
        printf("未能输出图中所有顶点，图中存在环。\\n");
}
```

Experiment 7-4 Graph topological sorting

Problems:

```
void DFS1(ALGraph G, int v); //邻接表实现  
void DFSTraverse1(ALGraph G); //邻接表实现  
void BFSTraverse(ALGraph *G); //邻接表实现  
void CreatGraph_DN(ALGraph *G);  
void TopologicalSort1(ALGraph G);  
void CountVerNum_DN(ALGraph *G, int D[]);
```


二、关键路径

1、**问题提出**：把工程计划表示为有向图，用顶点表示事件，弧表示活动；

例如：设一个工程有11项活动，9个事件

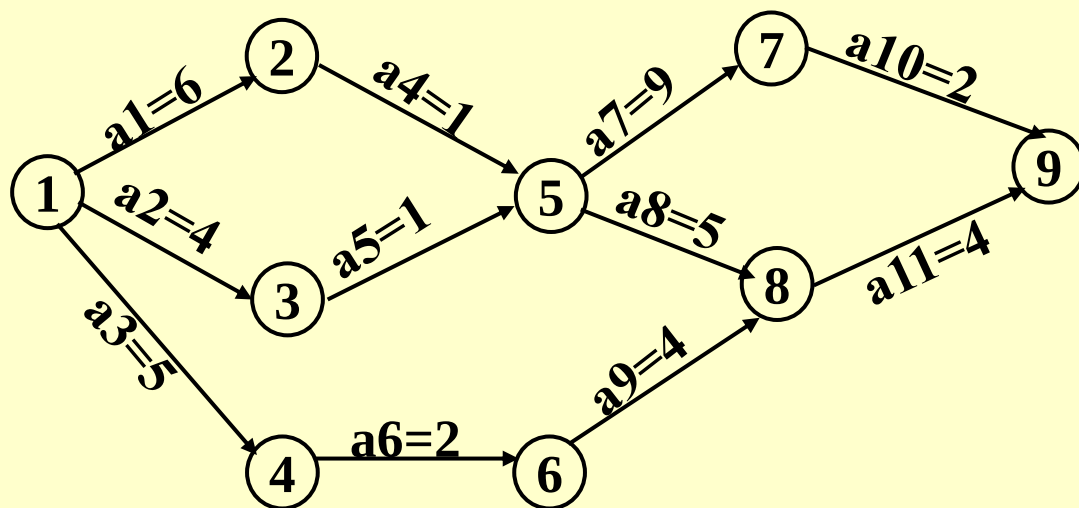
事件 V_1 ——表示整个工程开始

事件 V_9 ——表示整个工程结束

V_5 表示 a_4 、 a_5 已经完成， a_7 、 a_8 可以开始。

与每个活动相联系的数是执行该活动所需时间。

问题：（1）完成整项工程至少需要多少时间？
（2）哪些活动是影响工程进度的关键？



- **AOE网(Activity On Edge)**——边表示活动的网，一个带权的有向无环图，其中顶点表示事件，弧表示活动，权表示活动持续时间

AOE网具有以下两个性质：

①只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始。

②只有在进入一某顶点的各有向边所代表的活动都已经结束，该顶点所代表的事件才能发生。

- **路径长度**——路径上各活动持续时间之和
- **关键路径**——路径长度最长的路径，如(V1,V2,V5,V8,V9)
- **$Ve(j)$** ——表示事件 V_j 的最早发生时间, 指从源点到顶点 j 的最大路径长度代表的时间。 (V1,V2,V5,V8,V9),18 $Ve(9)=18$
- **$Vl(j)$** ——表示事件 V_j 的最迟发生时间,是指在不推迟整个工期的前提下，事件 v_j 允许的最晚发生时间。 $Vl(7)=16$
- **$e(i)$** ——表示活动 a_i 的最早开始时间. $e(6)=5$
- **$l(i)$** ——表示活动 a_i 的最迟开始时间。 $l(6)=8$ 18-a11-a9-a6
- **$l(i)-e(i)$** ——表示完成活动 a_i 的时间余量。
- **关键活动**——关键路径上的活动，即 $l(i)=e(i)$ 的活动



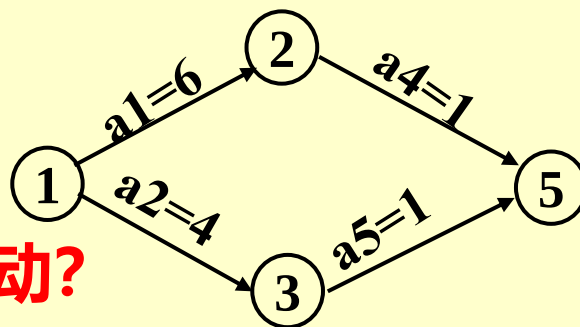
利用AOE网进行工程管理时解决的主要问题是：

- **计算完成整个工程的最短路径。**
- **确定关键路径，找出哪些活动是影响工程进度的关键。**

2、关键路径的确定

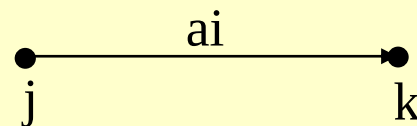
1) 问题分析

➤ 如何找 $e(i)=l(i)$ 的关键活动?



设活动 a_i 用弧 $\langle j, k \rangle$ 表示, 其持续时间记为: $dut(\langle j, k \rangle)$

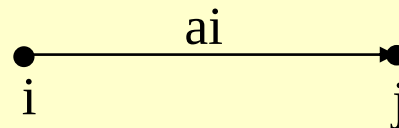
则: (1) $e(i) = Ve(j)$



(2) $l(i) = Vl(k) - dut(\langle j, k \rangle)$

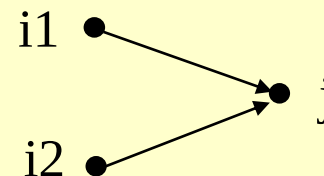
➤ 如何求 $Ve(j)$ 和 $Vl(j)$?

(1) 从 $Ve(1)=0$ 开始正向递推(i 变化)



$$Ve(j) = \underset{i}{Max}\{Ve(i) + dut(\langle i, j \rangle)\}, \langle i, j \rangle \in T, 2 \leq j \leq n$$

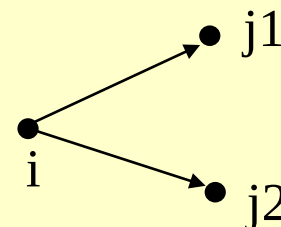
其中 T 是所有以 j 为头的弧的集合



(2) 从 $Vl(n)=Ve(n)$ 开始反向递推(j 变化)

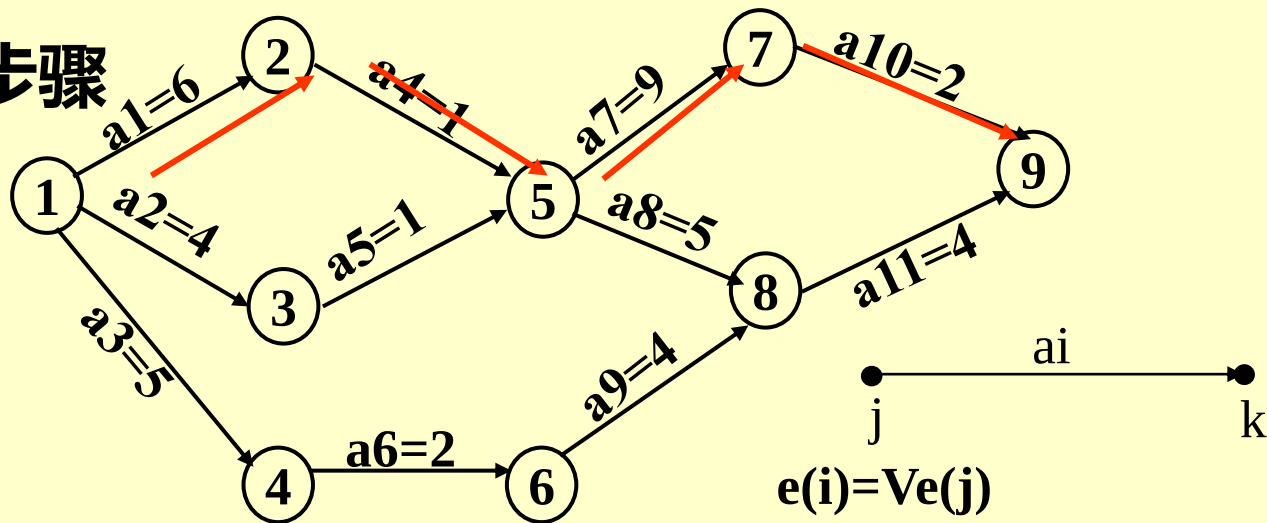
$$Vl(i) = \underset{j}{Min}\{Vl(j) - dut(\langle i, j \rangle)\}, \langle i, j \rangle \in S, 1 \leq i \leq n-1$$

其中 S 是所有以 i 为尾的弧的集合



2)、求关键路径步骤

- 求 $Ve(i)$
- 求 $Vl(j)$
- 求 $e(i)$
- 求 $l(i)$
- 计算 $l(i)-e(i)$



$$e(i) = Ve(j)$$

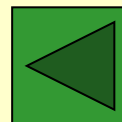
$$l(i) = Vl(k) - \text{dut}(<j, k>)$$

顶点	Ve	Vl
V1	0	0
V2	6	6
V3	4	6
V4	5	8
V5	7	7
V6	7	10
V7	16	16
V8	12	14
V9	18	18

9?

活动	e	l	l-e
a1	0	0	0 ✓
a2	0	2	2
a3	0	3	3
a4	6	6	0 ✓
a5	4	6	2
a6	5	8	3
a7	7	7	0 ✓
a8	7	9	2
a9	7	10	3
a10	16	16	0 ✓
a11	12	14	2

如果 $a8=7$?



3)、算法实现

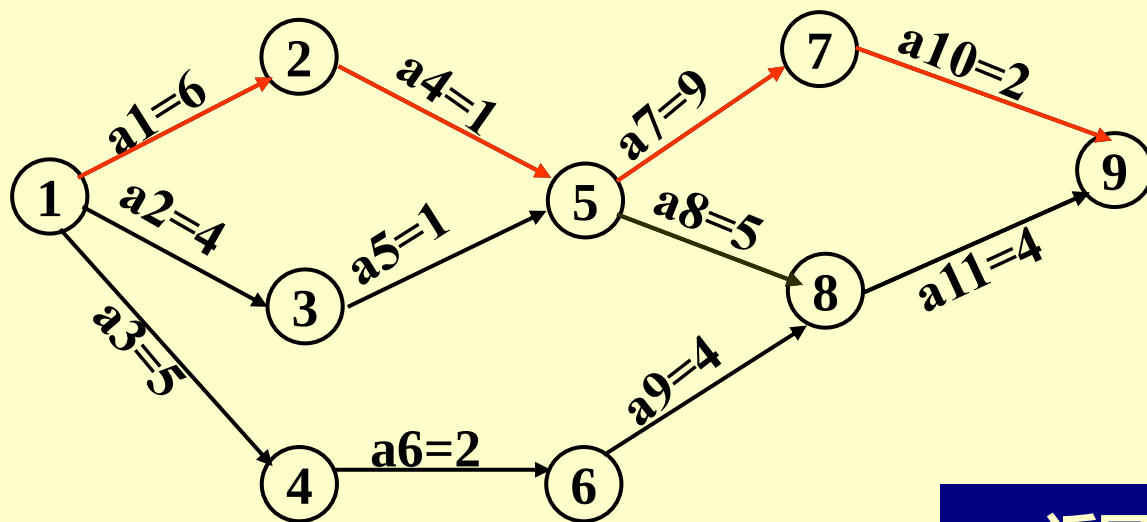
- 以邻接表作存储结构,输入e条弧<j,k>,建立AOE网 .
- 从源点V0出发, 令 $Ve[0]=0$,按拓扑序列求各顶点的 $Ve[i]$ ($1 \leq i \leq n-1$)。如果得到的拓扑有序序列中顶点个数小于网中顶点数n, 则说明网中存在环, 不能求关键路径, 算法终止; 否则执行步骤 (3) 。
- 从汇点 V_n 出发, 令 $VI[n-1]=Ve[n-1]$,按逆拓扑序列求其余各顶点的 $VI[i]$ ($n-2 \geq i \geq 2$)
- 根据各顶点的 Ve 和 VI 值, 计算每条弧的 $e[i]$ 和 $l[i]$,找出 $e[i]=l[i]$ 的关键活动

```
typedef struct arcnode    // 表结点
{ int adjvex;
  struct arcnode *nextarc;
  infotype      *info;
}arcnode;
```

adjvex	next	info
--------	------	------

● 算法描述

- 输入顶点和弧信息，建立其邻接表
- 计算每个顶点的入度
- 对其进行拓扑排序
 - 排序过程中求顶点的 $Ve[i]$
 - 将得到的拓扑序列进栈
- 按逆拓扑序列求顶点的 $VI[i]$
- 计算每条弧的 $e[i]$ 和 $l[i]$,找出 $e[i]=l[i]$ 的关键活动



返回

Experiment 7-5 Critical Path

Problems:

void CreatGraph_DN(ALGraph *G);

int TopologicalSortPath(ALGraph G, Stack T);

void CriticalPath(ALGraph G, Stack T);

void VistGraph(VertexNode V);

void DFS(ALGraph G, int v);

void CountVerNum_DN(ALGraph *G, int D[]);

Experiment 7-6 邻接表实现图所有的功能

Problems:

```
printf("-----\n");
printf("    图的邻接表实现相关算法\n");
printf("=====\n");
printf("1. 创建一个图;    2. 深度优先搜索;\n");
printf("3. 广度优先搜索;  4. 拓扑排序;\n");
printf("5. 关键路径计算;  6. 顶点度计算;\n");
printf("0. Exit;\n");
printf("=====\n");
```

图中的最短路径问题

在交通咨询系统中，用带权的有向图表示交通运输网，图中：

- 顶点——表示城市
- 边——表示城市间的交通联系
- 权——表示此线路的长度或沿此线路运输所花的时间或费用

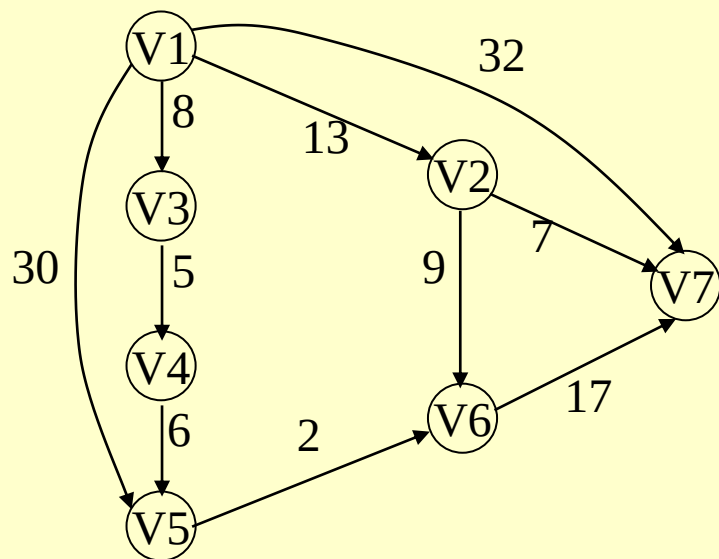
问题1：从某顶点出发，沿图的边到达另一顶点所经过的路径中，各边上权值之和最小的一条路径——最短路径。迪杰斯特拉(Dijkstra)

- 路径上的第一个顶点为源点 (Source)
- 最后一个顶点为终点 (Destination)

问题2：每一对顶点之间的最短路径。弗洛伊德(Floyd)算法

一、从某个源点到其余各顶点的最短路径

1、问题描述：给定带权有向图G和源点v，求从v到G中其余各顶点的最短路径。



最短路径	长度
<V1,V2>	13
<V1,V3>	8
<V1,V3,V4>	13
<V1,V3,V4,V5>	19
<V1,V3,V4,V5,V6>	21
<V1,V2,V7>	20

2、迪杰斯特拉(Dijkstra)算法思想

1) 基本思想：按路径长度递增次序产生最短路径算法。

把V分成两组：

(1) S：已求出最短路径的顶点的集合

(2) $V-S=T$ ：尚未确定最短路径的顶点集合

将T中顶点按最短路径递增的次序加入到S中，保证：

(a) 从源点 V_0 到S中各顶点的最短路径长度都不大于从 V_0 到T中任何顶点的最短路径长度

(b) 每个顶点对应一个距离值

S中顶点：从 V_0 到此顶点的最短路径长度

T中顶点：从 V_0 到此顶点的只包括S中顶点作中间顶点的最短路径长度

依据：可以证明 V_0 到T中顶点 V_k 的最短路径，或是从 V_0 到 V_k 的直接路径的权值；或是从 V_0 经S中顶点到 V_k 的路径权值之和（反证法可证）。

最短路径的最优子结构性质：

如果 $P(i,j)=\{V_i....V_k..V_s...V_j\}$ 是从顶点 i 到 j 的最短路径， k 和 s 是这条路径上的一个中间顶点，那么 $P(k,s)$ 必定是从 k 到 s 的最短路径。

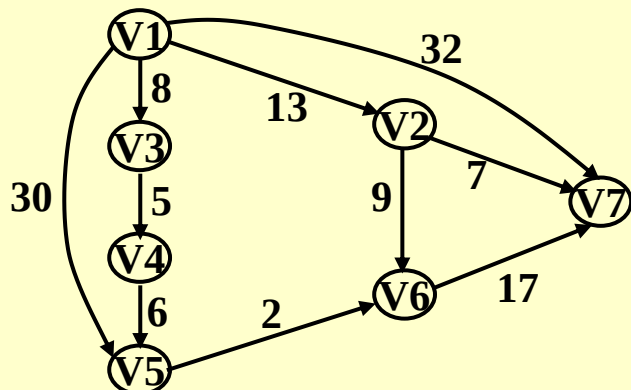
证明：

假设 $P(i,j)=\{V_i....V_k..V_s...V_j\}$ 是从顶点 i 到 j 的最短路径，则有 $P(i,j)=P(i,k)+P(k,s)+P(s,j)$ 。而 $P(k,s)$ 不是从 k 到 s 的最短距离，那么必定存在另一条从 k 到 s 的最短路径 $P'(k,s)$ ，那么 $P'(i,j)=P(i,k)+P'(k,s)+P(s,j)<P(i,j)$ 。则与 $P(i,j)$ 是从 i 到 j 的最短路径相矛盾。得证。

2) 求最短路径步骤

- 初使时令 $S=\{V_0\}$, $T=\{\text{其余顶点}\}$, T 中顶点对应的距离值
 - 若存在 $\langle V_0, V_i \rangle$, 为 $\langle V_0, V_i \rangle$ 弧上的权值
 - 若不存在 $\langle V_0, V_i \rangle$, 为 ∞
- 从 T 中选取一个其距离值为最小的顶点 u , 加入 S
- 对 T 中顶点的距离值进行修改: 若加进 u 作中间顶点, 从 V_0 到 V_i 的距离值比不加 u 的路径要短, 则修改此距离值
- 重复上述步骤, 直到 S 中包含所有顶点, 即 $S=V$ 为止





终点	从V1到各终点的最短路径及其长度				
V2	13 <V1,V2>	13 <V1,V2>	-----	-----	-----
V3	8 <V1,V3>	-----	-----	-----	-----
V4	∞	13 <V1,V3,V4>	13 <V1,V3,V4>	-----	-----
V5	30 <V1,V5>	30 <V1,V5>	30 <V1,V5>	19 <V1,V3,V4,V5>	-----
V6	∞	∞	22 <V1,V2,V6>	22 <V1,V2,V6>	21 <V1,V3,V4,V5,V6>
V7	32 <V1,V7>	32 <V1,V7>	20 <V1,V2,V7>	20 <V1,V2,V7>	20 <V1,V2,V7>
Vj	V3:8 <V1,V3>	V2:13 <V1,V2>	V4:13 <V1,V3,V4>	V5:19 <V1,V3,V4,V5>	V7:20 <V1, V2,V7>

3) 算法实现

- 图用带权邻接矩阵存储Edge[][]

Edge [i][j] 表示弧 $\langle v_i, v_j \rangle$ 上的权值。若 $\langle v_i, v_j \rangle$ 不存在，则置Edge [i][j]为 ∞ （在计算机上可用允许的最大值代替）。

- 数组dist[]:

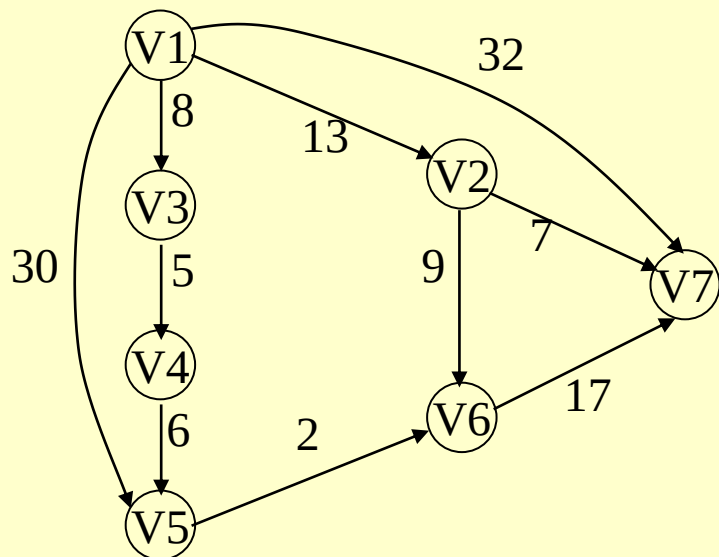
存放当前找到的从源点到每个终点的最短路径长度，其初态为图中直接路径权值

- 数组pre[]:

表示从源点到各终点的最短路径上，此顶点的前一顶点的序号；若从源点到某终点无路径，则用-1作为其前一顶点的序号



● 算法描述



最短路径	长度
<V1,V2>	13
<V1,V3>	8
<V1,V3,V4>	13
<V1,V3,V4,V5>	19
<V1,V3,V4,V5,V6>	21
<V1,V2,V7>	20

Edge[][] =

∞	13	8	∞	30	∞	32
	∞	∞	∞	∞	9	7
		∞	5	∞	∞	∞
			∞	6	∞	∞
				∞	2	∞
					∞	17
						∞

	0	1	2	3	4	5	6	
dist	0	13	8	13	19	21	20	min=8
								u=2
	0	1	2	3	4	5	6	
pre	-1	0	0	2	3	4	1	k=3
								dist[3]=min
vexs	V1	V2	V3	V4	V5	V6	V7	+Edge[u][k]
visited	F	F	F	F	F	F	F	

● 算法分析: $T(n)=O(n^2)$

由pre数组如何获得并输出各最短径?

Experiment 7-7 Dijkstra for shortest path

Problems:

void CreateGraph(Graph *G);

int locate(Graph *G, VertexType data);

void DijkstraPath(Graph G, int *dist, int *pre, int v0);

int showPath(Graph *G, int *pre, int v0, int v);

二、每一对顶点之间的最短路径

1、方法一：每次以一个顶点为源点，重复执行

Dijkstra算法n次—— $T(n)=O(n^3)$

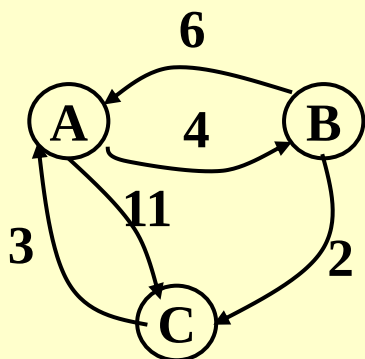
2、方法二：弗洛伊德(Floyd)算法

— 算法思想：逐个顶点试探法

— 求最短路径步骤

- 初始时设置一个n阶方阵，令其对角线元素为0，若存在弧 $\langle V_i, V_j \rangle$ ，则对应元素为权值；否则为 ∞
- 逐步试着在原直接路径中增加中间顶点，若加入中间点后路径变短，则修改之；否则，维持原值
- 所有顶点试探完毕，算法结束。

例



初始:
$$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$$

路径:

	AB	AC
BA		BC
CA		

加入A:
$$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

路径:

	AB	AC
BA		BC
CA	CAB	

加入B:
$$\begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

路径:

	AB	ABC
BA		BC
CA	CAB	

加入C:
$$\begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

路径:

	AB	ABC
BCA		BC
CA	CAB	

— 算法实现:

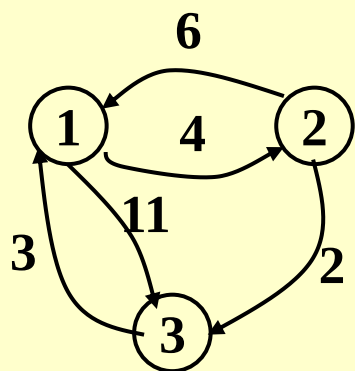
- 图用邻接矩阵存储
- length[][]存放最短路径长度
- path[i][j]是从Vi到Vj的最短路径上Vj前一顶点序号

— 算法描述:P191

初始:

$$\text{length} = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix} \quad \text{path} = \begin{pmatrix} 0 & 1 & 1 \\ 2 & 0 & 2 \\ 3 & 0 & 0 \end{pmatrix}$$

例



加入A:

$$\text{length} = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad \text{path} = \begin{pmatrix} 0 & 1 & 1 \\ 2 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix}$$

加入B:

$$\text{length} = \begin{pmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad \text{path} = \begin{pmatrix} 0 & 1 & 2 \\ 2 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix}$$

加入C:

$$\text{length} = \begin{pmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix} \quad \text{path} = \begin{pmatrix} 0 & 1 & 2 \\ 3 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix}$$

- path数组与pre数组区别?
- 由path数组如何获得并输出各最短径?

Experiment 7-8 Floyd for shortest path

Problems:

void CreateGraph(Graph *G);

int locate(Graph *G, VertexType data);

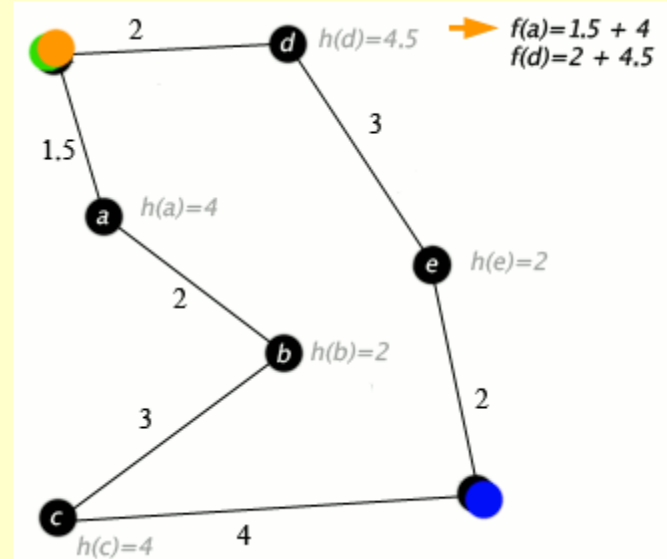
void Floyd(Graph g, int **path, int **length);

int showPath(Graph *g, int **path, int v0, int v);

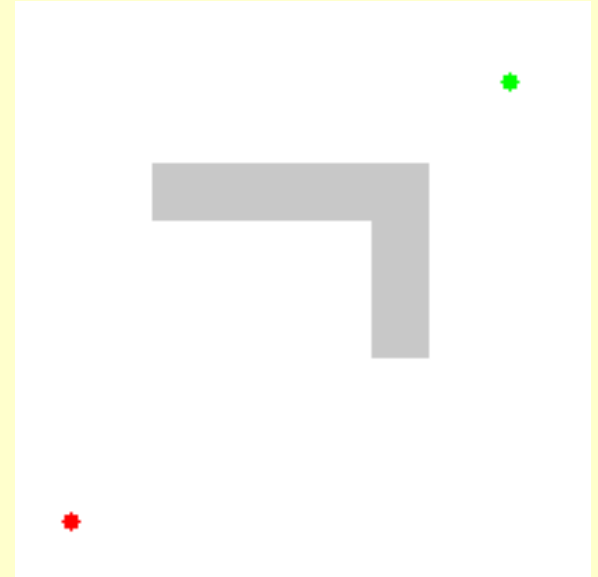
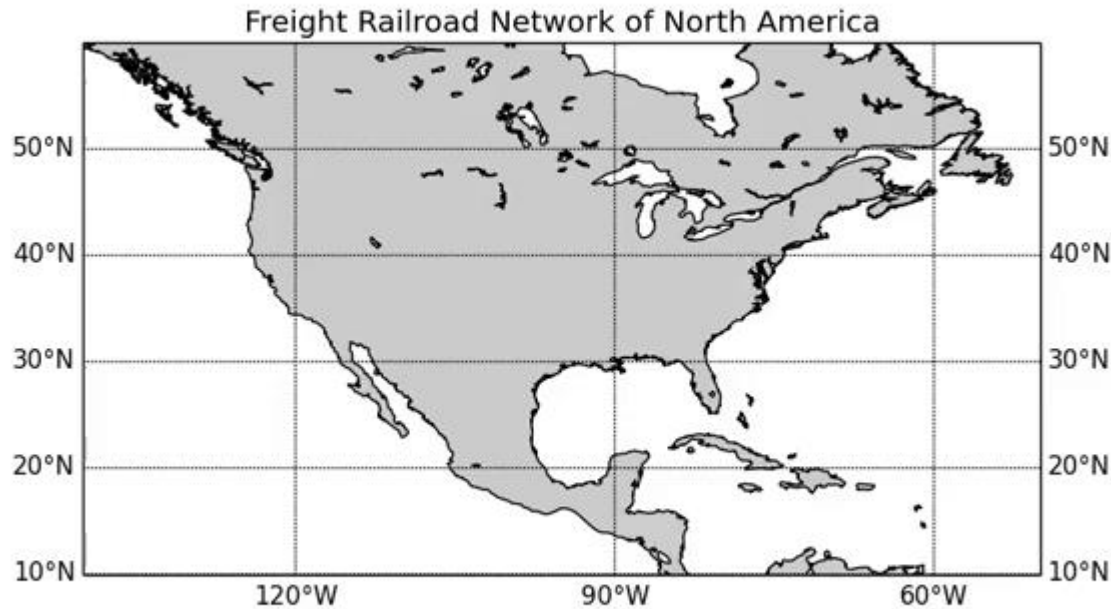
A* search algorithm

Peter Hart, Nils Nilsson and Bertram Raphael of Stanford Research Institute, 1968, an extension of Edsger Dijkstra's 1959 algorithm.

- A* is a **best-first search** by searching among all possible paths to the solution (goal) with the smallest cost (least distance travelled, shortest time, etc.).
- At each iteration of its main loop, A* needs to determine which of its partial paths to expand into one or more longer paths. It does so based on an estimate of the cost (total weight) still to go to the goal node. Specifically, A* selects the path that minimizes $f(n)=g(n)+h(n)$
- where n is the last node on the path, $g(n)$ is the cost of the path from the start node to n , and $h(n)$ is a **heuristic** that estimates the cost of the cheapest path from n to the goal.
- A* use a **priority queue** to perform the repeated selection of minimum (estimated) cost nodes to expand.



应用



静态路径最短路径算法是外界环境不变，Dijkstra算法，A*算法。
动态路径最短路是外界环境不断发生变化，即不能计算预测的情况下计算最短路。如在游戏中敌人或障碍物不断移动的情况下。典型的有**D*算法**。

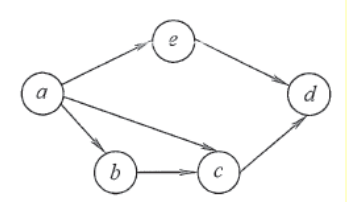
Experiment 7-9 shortest path

Problems:

- 1 合并两个算法;
- 2 .h文件使用。

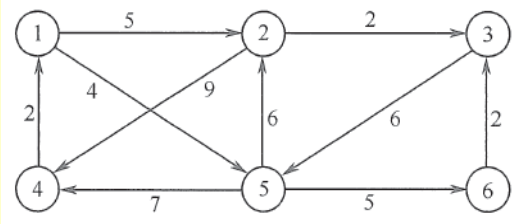
57. 【2010统考真题】对下图进行拓扑排序，可得不同拓扑序列的个数是（）。

- A、4 B、3 C、2 D、1



58. 【2016统考真题】使用Dijkstra算法求下图中从顶点1到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（）。

- A、5,2,3,4,6 B、5,2,3,6,4 C、5,2,4,3,6 D、5,2,6,3,4

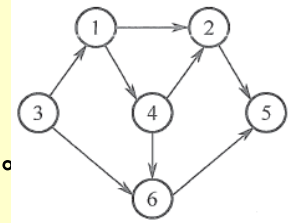


59. 【2016统考真题】对一个有n个顶点，e条弧的有向图采用邻接表存储时，则拓扑排序算法的时间复杂度是（）。

- A、O(n) B、O(n+e) C、O(n²) D、O(ne)

60. 在有向图G的拓扑序列中，若顶点vi在顶点vj之前，则下列情形不可能出现的是（）

- A、G中有弧<vi,vj> B、G中有一条从vi到vj的路径
C、G中没有弧<vi,vj> D、G中有一条从vj到vi的路径

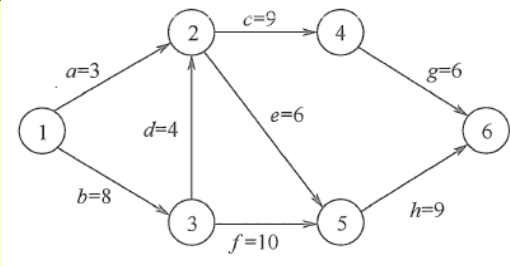


61. 【2014统考真题】对下图所示的有向图进行拓扑排序，得到的拓扑序列可能是（）。

- A、3,1,2,4,5,6 B、3,1,2,4,6,5 C、3,1,4,2,5,6 D、3,1,4,2,6,5

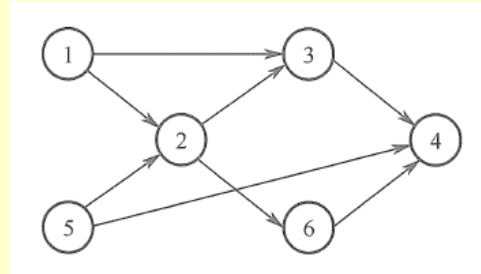
62. 【2013统考真题】下列AOE网表示一项包含9个活动的工程。通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（）。

- A、c和e B、d和c
C、f和d D、f和h



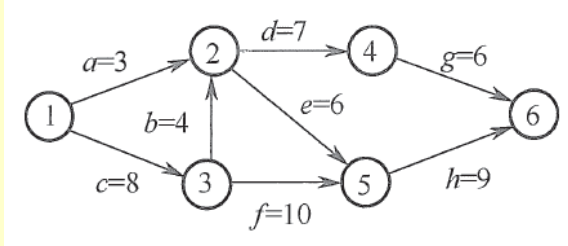
65. 【2018统考真题】下列选项中，不是如下图有向图的拓扑序列的是（）。

- A、1,5,2,3,6,4 B、5,1,2,6,3,4
C、5,1,2,3,6,4 D、5,2,1,6,3,4



66. 【2019统考真题】下图所示的AOE网表示一项包含8个活动的工程。活动d的最早开始时间和最迟开始时间分别是（）。

- A、3和7 B、12和12
C、12和14 D、15和15

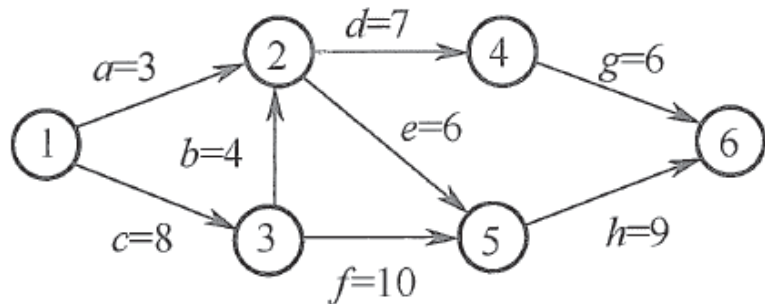


66. 下图所示的AOE网表示一项包含8个活动的工程。

活动d的最早开始时间和最迟开始时间分别是（ ）。

A、3和7 B、12和12

C、12和14 D、15和15



$$e(i)=Ve(j)$$

$$l(i)=Vl(k)-dut(<j,k>)$$

$$Ve(j) = \underset{i}{Max}\{Ve(i) + dut(<i,j>)\}, <i,j> \in T, 2 \leq j \leq n$$

其中T是所有以j为头的弧的集合

$$Vl(i) = \underset{j}{Min}\{Vl(j) - dut(<i,j>)\}, <i,j> \in S, 1 \leq i \leq n-1$$

其中S是所有以i为尾的弧的集合